III

Advanced Multilayer Perceptrons and Keras

Use Keras Deep Learning Models with scikit-learn

13

Keras is one of the most popular deep learning libraries in Python for research and development because of its simplicity and ease of use. The scikit-learn library is the most popular library for general machine learning in Python. In this chapter, you will discover how to use deep learning models from Keras with the scikit-learn library in Python. This will allow you to leverage the power of the scikit-learn library for tasks like model evaluation and model hyper-parameter optimization. After completing this lesson you will know:

- ▷ How to wrap a Keras model for use with the scikit-learn machine learning library
- ▷ How to easily evaluate Keras models using cross-validation in scikit-learn
- ▷ How to tune Keras model hyperparameters using grid search in scikit-learn

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Overview of SciKeras
- ▷ Evaluate Deep Learning Models with Cross-Validation
- ▷ Grid Search Deep Learning Model Parameters

13.1 Overview of SciKeras

Keras is a popular library for deep learning in Python, but the focus of the library is *deep learning*, not all of machine learning. In fact, it strives for minimalism, focusing on only what you need to quickly and simply define and build deep learning models. The scikit-learn library in Python is built upon the SciPy stack for efficient numerical computation. It is a fully featured library for general purpose machine learning and provides many useful utilities in developing deep learning models. Not least of which are:

- ▷ Evaluation of models using resampling methods like k-fold cross-validation
- ▷ Efficient search and evaluation of model hyperparameters

There was a wrapper in the TensorFlow/Keras library to make deep learning models used as classification or regression estimators in scikit-learn. But recently, this wrapper was taken out to become a standalone Python module.

In the following sections, you will work through examples of using the `KerasClassifier` wrapper for a classification neural network created in Keras and used in the scikit-learn library. The test problem is the Pima Indians onset of diabetes classification dataset (see Section 7.1). This is a small dataset with all numerical attributes that is easy to work with.

The following examples assume you have successfully installed TensorFlow 2.x, SciKeras, and scikit-learn. If you use the `pip` for your Python modules, you may install them with:

```
pip install tensorflow scikeras scikit-learn
```

Listing 13.1: Installing required libraries

13.2 Evaluate Deep Learning Models with Cross-Validation

The `KerasClassifier` and `KerasRegressor` classes in SciKeras take an argument `model` which is the name of the function to call to get your model. You must define a function that defines your model, compiles it, and returns it. In the example below, you will define a function `create_model()` that creates a simple multilayer neural network for the problem.

You pass this function name to the `KerasClassifier` class by the `model` argument. You also pass in additional arguments of `epoch=150` and `batch_size=10`. These are automatically bundled up and passed on to the `fit()` function, which is called internally by the `KerasClassifier` class. In this example, you will use the scikit-learn `StratifiedKFold` to perform 10-fold stratified cross-validation. This is a resampling technique that can provide a robust estimate of the performance of a machine learning model on unseen data. Next, use the scikit-learn function `cross_val_score()` to evaluate your model using cross-validation and print the results.

```
# MLP for Pima Indians Dataset with 10-fold cross validation via sklearn
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
import numpy as np

# Function to create model, required for KerasClassifier
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(8, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

```
# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=150, batch_size=10, verbose=0)
# evaluate using 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=seed)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Listing 13.2: Evaluate a neural network using scikit-learn

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example displays the skill of the model for each epoch. A total of 10 models are created and evaluated, and the final average accuracy is displayed.

```
0.646838691487
```

Output 13.1: The final average accuracy evaluated

You can see that when the Keras model is wrapped that estimating model accuracy can be greatly streamlined, compared to the manual enumeration of cross-validation folds performed in the previous chapter.

In comparison, the following is an equivalent implementation with a neural network model in scikit-learn:

```
# MLP for Pima Indians Dataset with 10-fold cross validation via sklearn
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
from sklearn.neural_network import MLPClassifier
import numpy as np

# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = MLPClassifier(hidden_layer_sizes=(12,8), activation='relu',
```

```
                    max_iter=150, batch_size=10, verbose=False)
# evaluate using 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=seed)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Listing 13.3: Equivalent implementation in scikit-learn

The role of the `KerasClassifier` is to work as an adapter to make the Keras model work like a `MLPClassifier` object from scikit-learn.

13.3 Grid Search Deep Learning Model Parameters

The previous example showed how easy it is to wrap your deep learning model from Keras and use it in functions from the scikit-learn library. In this example, you will go a step further. The function that you specify to the `model` argument when creating the `KerasClassifier` wrapper can take arguments. You can use these arguments to further customize the construction of the model. In addition, you know you can provide arguments to the `fit()` function.

In this example, you will use grid search to evaluate different configurations for your neural network model and report on the combination that provides the best estimated performance. The `create_model()` function is defined to take two arguments, `optimizer` and `init`, both of which must have default values. This will allow you to evaluate the effect of using different optimization algorithms and weight initialization schemes for your network. After creating your model, you define the arrays of values for the parameter you wish to search, specifically:

- ▷ Optimizers for searching different weight values
- ▷ Initializers for preparing the network weights using different schemes
- ▷ Number of epochs for training the model for different number of exposures to the training dataset
- ▷ Batches for varying the number of samples before weight updates

The options are specified into a dictionary and passed to the configuration of the `GridSearchCV` scikit-learn class. This class will evaluate a version of your neural network model for each combination of parameters ($2 \times 3 \times 3 \times 3$ for the combinations of optimizers, initializations, epochs, and batches). Each combination is then evaluated using the default of 3-fold stratified cross-validation.

That is a lot of models and a lot of computation. This is not a scheme you want to use lightly because it takes a long time to compute. It may be useful for you to design small models with a smaller subset of data that will complete in a reasonable time. It is the case of this experiment (less than 1000 instances and nine attributes). Finally, the performance and combination of configurations for the best model are displayed, followed by the performance of all combinations of parameters.

```
# MLP for Pima Indians Dataset with grid search via sklearn
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import GridSearchCV
import numpy as np

# Function to create model, required for KerasClassifier
def create_model(optimizer='rmsprop', init='glorot_uniform'):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), kernel_initializer=init, activation='relu'))
    model.add(Dense(8, kernel_initializer=init, activation='relu'))
    model.add(Dense(1, kernel_initializer=init, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])
    return model

# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, verbose=0)
print(model.get_params().keys())
# grid search epochs, batch size and optimizer
optimizers = ['rmsprop', 'adam']
init = ['glorot_uniform', 'normal', 'uniform']
epochs = [50, 100, 150]
sizes = [5, 10, 20]
param_grid = dict(optimizer=optimizers, epochs=epochs, batch_size=sizes, model__init=init)
grid = GridSearchCV(estimator=model, param_grid=param_grid)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 13.4: Grid Search Neural Network Parameters Using scikit-learn

Note that in the dictionary `param_grid`, `model__init` was used as the key for the `init` argument to our `create_model()` function. The prefix `model__` is required for `KerasClassifier` models in SciKeras to provide custom arguments.

This might take about 5 minutes to complete on your workstation executed on the CPU (rather than GPU). Running the example shows the results below. You can see that the grid search discovered that using a uniform initialization scheme, rmsprop optimizer, 150 epochs, and a batch size of 10 achieved the best cross-validation score of approximately 77% on this problem.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Best: 0.772167 using {'batch_size': 10, 'epochs': 150, 'model__init': 'normal', 'optimizer': 'rmspro
0.691359 (0.024495) with: {'batch_size': 5, 'epochs': 50, 'model__init': 'glorot_uniform', 'optimize
0.690094 (0.026691) with: {'batch_size': 5, 'epochs': 50, 'model__init': 'glorot_uniform', 'optimize
0.704482 (0.036013) with: {'batch_size': 5, 'epochs': 50, 'model__init': 'normal', 'optimizer': 'rms
0.727884 (0.016890) with: {'batch_size': 5, 'epochs': 50, 'model__init': 'normal', 'optimizer': 'ada
0.709600 (0.030281) with: {'batch_size': 5, 'epochs': 50, 'model__init': 'uniform', 'optimizer': 'rm
...
```

Output 13.2: Output of grid search neural network parameters using scikit-learn

13.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

SciKeras documentation.
https://www.adriangb.com/scikeras/stable/

Stratified K-Folds cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

Grid search cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

13.5 Summary

In this chapter, you discovered how to wrap your Keras deep learning models and use them in the scikit-learn general machine learning library. You learned:

- ▷ Specifically how to wrap Keras models so that they can be used with the scikit-learn machine learning library.
- ▷ How to use a wrapped Keras model as part of evaluating model performance in scikit-learn.
- ▷ How to perform hyperparameter tuning in scikit-learn using a wrapped Keras model.

You can see that using scikit-learn for standard machine learning operations such as model evaluation and model hyperparameter optimization can save a lot of time over implementing these schemes yourself. Wrapping your model allowed you to leverage powerful tools from scikit-learn to fit your deep learning models into your general machine learning process. We will see more examples in the next chapter.

How to Grid Search Hyperparameters for Deep Learning Models

14

Hyperparameters are the parameters of a neural network that is fixed by design and not tuned by training. Examples are the number of hidden layers and the choice of activation functions. Hyperparameter optimization is a big part of deep learning. The reason is that neural networks are notoriously difficult to configure, and a lot of parameters need to be set. On top of that, individual models can be very slow to train.

In this chapter, you will discover how to use the grid search capability from the scikit-learn Python machine learning library to tune the hyperparameters of Keras deep learning models. After reading this chapter, you will know:

- ▷ How to wrap Keras models for use in scikit-learn and how to use grid search
- ▷ How to grid search common neural network parameters, such as learning rate, dropout rate, epochs, and number of neurons
- ▷ How to define your own hyperparameter tuning experiments on your own projects

Let's get started.

Overview

In this chapter, I want to show you both how you can use the scikit-learn grid search capability and give you a suite of examples that you can copy-and-paste into your own project as a starting point. Below is a list of the topics we are going to cover:

- ▷ How to use Keras models in scikit-learn
- ▷ How to use grid search in scikit-learn
- ▷ How to tune batch size and training epochs
- ▷ How to tune optimization algorithms
- ▷ How to tune learning rate and momentum
- ▷ How to tune network weight initialization
- ▷ How to tune activation functions
- ▷ How to tune dropout regularization

▷ How to tune the number of neurons in the hidden layer

14.1 How to Use Keras Models in scikit-learn

Keras models can be used in scikit-learn by wrapping them with the `KerasClassifier` or `KerasRegressor` class from the module SciKeras. You may need to run the command in Listing 13.1 to install the module.

To use these wrappers, you must define a function that creates and returns your Keras sequential model, then pass this function to the `model` argument when constructing the `KerasClassifier` class. For example:

```
def create_model():
    ...
    return model

model = KerasClassifier(model=create_model)
```

Listing 14.1: Using KerasClassifier wrapper

The constructor for the `KerasClassifier` class can take default arguments that are passed on to the calls to `model.fit()`, such as the number of epochs and the batch size. For example:

```
def create_model():
    ...
    return model

model = KerasClassifier(model=create_model, epochs=10)
```

Listing 14.2: Using KerasClassifier with epochs argument

The constructor for the `KerasClassifier` class can also take new arguments that can be passed to your custom `create_model()` function. These new arguments must also be defined in the signature of your `create_model()` function with default parameters. For example:

```
def create_model(dropout_rate=0.0):
    ...
    return model

model = KerasClassifier(model=create_model, dropout_rate=0.2)
```

Listing 14.3: Using KerasClassifier with dropout rate argument

You can learn more about these from the SciKeras documentation.

14.2 How to Use Grid Search in scikit-learn

Grid search is a model hyperparameter optimization technique. It simply exhaust all combinations of the hyperparameters and find the one that gave the best score. In scikit-learn, this technique is provided in the `GridSearchCV` class. When constructing this class, you must

provide a dictionary of hyperparameters to evaluate in the `param_grid` argument. This is a map of the model parameter name and an array of values to try.

By default, accuracy is the score that is optimized, but other scores can be specified in the `score` argument of the `GridSearchCV` constructor. By default, the grid search will only use one thread. By setting the `n_jobs` argument in the `GridSearchCV` constructor to -1, the process will use all cores on your machine. However, sometimes this may interfere with the main neural network training process. The `GridSearchCV` process will then construct and evaluate one model for each combination of parameters. Cross-validation is used to evaluate each individual model, and the default of 3-fold cross-validation is used, although you can override this by specifying the `cv` argument to the `GridSearchCV` constructor.

Below is an example of defining a simple grid search:

```
param_grid = dict(epochs=[10,20,30])
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
```

Listing 14.4: Running grid search using scikit-learn

Once completed, you can access the outcome of the grid search in the result object returned from `grid.fit()`. The `best_score_` member provides access to the best score observed during the optimization procedure, and the `best_params_` describes the combination of parameters that achieved the best results. You can learn more about the `GridSearchCV` class in the scikit-learn API documentation.

14.3 Problem Description

Now that you know how to use Keras models with scikit-learn and how to use grid search in scikit-learn, let's look at a bunch of examples.

All examples will be demonstrated on a small standard machine learning dataset called the Pima Indians onset of diabetes classification dataset. This is a small dataset with all numerical attributes that is easy to work with. (See Section 7.1) As you proceed through the examples in this chapter, you will aggregate the best parameters. This is not the best way to grid search because parameters can interact, but it is good for demonstration purposes.

Note on Parallelizing Grid Search

All examples are configured to use parallelism (`n_jobs=-1`). If you get an error like the one below:

```
INFO (theano.gof.compilelock): Waiting for existing lock by process '55614' (I am process '55613')
INFO (theano.gof.compilelock): To manually release the lock, delete ...
```

Output 14.1: Error message due to parallelism

Kill the process and change the code to not perform the grid search in parallel; set `n_jobs=1`.

14.4 How to Tune Batch Size and Number of Epochs

In this first simple example, you will look at tuning the batch size and number of epochs used when fitting the network.

The batch size in iterative gradient descent is the number of patterns shown to the network before the weights are updated. It is also an optimization in the training of the network, defining how many patterns to read at a time and keep in memory. The number of epochs is the number of times the entire training dataset is shown to the network during training. Some networks are sensitive to the batch size, such as LSTM recurrent neural networks and Convolutional Neural Networks. Here you will evaluate a suite of different minibatch sizes from 10 to 100 in steps of 20.

The full code listing is provided below:

```
# Use scikit-learn to grid search the batch size and epochs
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, verbose=0)
# define the grid search parameters
batch_size = [10, 20, 40, 60, 80, 100]
epochs = [10, 50, 100]
param_grid = dict(batch_size=batch_size, epochs=epochs)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
```

```
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.5: Grid search on batch size and number of epochs

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running this example produces the following output:

```
Best: 0.705729 using {'batch_size': 10, 'epochs': 100}
0.597656 (0.030425) with: {'batch_size': 10, 'epochs': 10}
0.686198 (0.017566) with: {'batch_size': 10, 'epochs': 50}
0.705729 (0.017566) with: {'batch_size': 10, 'epochs': 100}
0.494792 (0.009207) with: {'batch_size': 20, 'epochs': 10}
0.675781 (0.017758) with: {'batch_size': 20, 'epochs': 50}
0.683594 (0.011049) with: {'batch_size': 20, 'epochs': 100}
0.535156 (0.053274) with: {'batch_size': 40, 'epochs': 10}
0.622396 (0.009744) with: {'batch_size': 40, 'epochs': 50}
0.671875 (0.019918) with: {'batch_size': 40, 'epochs': 100}
0.592448 (0.042473) with: {'batch_size': 60, 'epochs': 10}
0.660156 (0.041707) with: {'batch_size': 60, 'epochs': 50}
0.674479 (0.006639) with: {'batch_size': 60, 'epochs': 100}
0.476562 (0.099896) with: {'batch_size': 80, 'epochs': 10}
0.608073 (0.033197) with: {'batch_size': 80, 'epochs': 50}
0.660156 (0.011500) with: {'batch_size': 80, 'epochs': 100}
0.615885 (0.015073) with: {'batch_size': 100, 'epochs': 10}
0.617188 (0.039192) with: {'batch_size': 100, 'epochs': 50}
0.632812 (0.019918) with: {'batch_size': 100, 'epochs': 100}
```

Output 14.2: Result of grid search on batch size and number of epochs

You can see that the batch size of 10 and 100 epochs achieved the best result of about 70% accuracy.

14.5 How to Tune the Training Optimization Algorithm

Keras offers a suite of different state-of-the-art optimization algorithms. In this example, you will tune the optimization algorithm used to train the network, each with default parameters. This is an odd example because often, you will choose one approach a priori and instead focus on tuning its parameters on your problem (see the next example). Here, you will evaluate the suite of optimization algorithms supported by the Keras API.

The full code listing is provided below:

```
# Use scikit-learn to grid search the optimization algorithms
import numpy as np
import tensorflow as tf
```

```
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # return model without compile
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, loss="binary_crossentropy",
                        epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
optimizer = ['SGD', 'RMSprop', 'Adagrad', 'Adadelta', 'Adam', 'Adamax', 'Nadam']
param_grid = dict(optimizer=optimizer)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.6: Grid search on optimization algorithms

Note the function `create_model()` defined above does not return a compiled model like that one in the previous example. This is because setting an optimizer for a Keras model is done in the `compile()` function call; hence it is better to leave it to the `KerasClassifier` wrapper and the `GridSearchCV` model. Also, note that you specified `loss="binary_crossentropy"` in the wrapper as it should also be set during the `compile()` function call.

Running this example produces the following output:

```
Best: 0.697917 using {'optimizer': 'Adam'}
0.674479 (0.033804) with: {'optimizer': 'SGD'}
0.649740 (0.040386) with: {'optimizer': 'RMSprop'}
0.595052 (0.032734) with: {'optimizer': 'Adagrad'}
0.348958 (0.001841) with: {'optimizer': 'Adadelta'}
0.697917 (0.038051) with: {'optimizer': 'Adam'}
0.652344 (0.019918) with: {'optimizer': 'Adamax'}
0.684896 (0.011201) with: {'optimizer': 'Nadam'}
```

Output 14.3: Result of grid search on optimization algorithms

The `KerasClassifier` wrapper will not compile your model again if the model is already compiled. Hence the other way to run `GridSearchCV` is to set the optimizer as an argument to the `create_model()` function, which returns an appropriately compiled model like the following:

```
# Use scikit-learn to grid search the optimization algorithms
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model(optimizer='adam'):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
optimizer = ['SGD', 'RMSprop', 'Adagrad', 'Adadelta', 'Adam', 'Adamax', 'Nadam']
param_grid = dict(model__optimizer=optimizer)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.7: Grid search on optimization algorithms with compiled model

Note that in the above, you have the prefix `model__` in the parameter dictionary `param_grid`. This is required for the `KerasClassifier` in the SciKeras module to make clear that the parameter needs to *route* into the `create_model()` function as arguments, rather than some parameter to set up in `compile()` or `fit()`. See also the routed parameter section of SciKeras documentation.

Running this example produces the following output:

```
Best: 0.697917 using {'model__optimizer': 'Adam'}
0.636719 (0.019401) with: {'model__optimizer': 'SGD'}
0.683594 (0.020915) with: {'model__optimizer': 'RMSprop'}
0.585938 (0.038670) with: {'model__optimizer': 'Adagrad'}
0.518229 (0.120624) with: {'model__optimizer': 'Adadelta'}
0.697917 (0.049445) with: {'model__optimizer': 'Adam'}
0.652344 (0.027805) with: {'model__optimizer': 'Adamax'}
0.686198 (0.012890) with: {'model__optimizer': 'Nadam'}
```

Output 14.4: Result of grid search on optimization algorithms, using compiled model

The results suggest that the ADAM optimization algorithm is the best with a score of about 70% accuracy.

14.6 How to Tune Learning Rate and Momentum

It is common to pre-select an optimization algorithm to train your network and tune its parameters. By far, the most common optimization algorithm is plain old Stochastic Gradient Descent (SGD) because it is so well understood. In this example, you will look at optimizing the SGD learning rate and momentum parameters.

The learning rate controls how much to update the weight at the end of each batch, and the momentum controls how much to let the previous update influence the current weight update. You will try a suite of small standard learning rates and a momentum values from 0.2 to 0.8 in steps of 0.2, as well as 0.9 (because it can be a popular value in practice). In Keras, the way to set the learning rate and momentum is the following:

```
...
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.2)
```

Listing 14.8: Setting learning rate and momentum in SGD

In the SciKeras wrapper, you will *route* the parameters to the optimizer with the prefix `optimizer__`. Generally, it is a good idea to also include the number of epochs in an optimization like this as there is a dependency between the amount of learning per batch (learning rate), the number of updates per epoch (batch size), and the number of epochs.

The full code listing is provided below:

```
# Use scikit-learn to grid search the learning rate and momentum
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model():
    # create model
    model = Sequential()
```

```
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, loss="binary_crossentropy",
                        optimizer="SGD", epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
learn_rate = [0.001, 0.01, 0.1, 0.2, 0.3]
momentum = [0.0, 0.2, 0.4, 0.6, 0.8, 0.9]
param_grid = dict(optimizer__learning_rate=learn_rate, optimizer__momentum=momentum)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.9: Grid search on learning rate and momentum

Running this example produces the following output.

```
Best: 0.686198 using {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.0}
0.686198 (0.036966) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.0}
0.651042 (0.009744) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.2}
0.652344 (0.038670) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.4}
0.656250 (0.065907) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.6}
0.671875 (0.022326) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.8}
0.661458 (0.015733) with: {'optimizer__learning_rate': 0.001, 'optimizer__momentum': 0.9}
...
```

Output 14.5: Result of grid search on learning rate and momentum

You can see that SGD is not very good on this problem; nevertheless, the best results were achieved using a learning rate of 0.001 and a momentum of 0.0 with an accuracy of about 68%.

14.7 How to Tune Network Weight Initialization

Neural network weight initialization used to be simple: use small random values. Now there is a suite of different techniques to choose from. Keras provides a laundry list. In this example, you will look at tuning the selection of network weight initialization by evaluating all the available techniques.

You will use the same weight initialization method on each layer. Ideally, it may be better to use different weight initialization schemes according to the activation function used on each layer. In the example below, you will use a rectifier for the hidden layer. Use sigmoid for the output layer because the predictions are binary. The weight initialization is now an argument to `create_model()` function, where you need to use the `model__` prefix to ask the `KerasClassifier` to route the parameter to the model creation function.

The full code listing is provided below:

```
# Use scikit-learn to grid search the weight initialization
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model(init_mode='uniform'):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), kernel_initializer=init_mode,
                    activation='relu'))
    model.add(Dense(1, kernel_initializer=init_mode, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
init_mode = ['uniform', 'lecun_uniform', 'normal', 'zero', 'glorot_normal',
             'glorot_uniform', 'he_normal', 'he_uniform']
param_grid = dict(model__init_mode=init_mode)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.10: Grid search on weight initialization

Running this example produces the following output.

```
Best: 0.716146 using {'model__init_mode': 'uniform'}
0.716146 (0.034987) with: {'model__init_mode': 'uniform'}
0.678385 (0.029635) with: {'model__init_mode': 'lecun_uniform'}
0.716146 (0.030647) with: {'model__init_mode': 'normal'}
0.651042 (0.001841) with: {'model__init_mode': 'zero'}
0.695312 (0.027805) with: {'model__init_mode': 'glorot_normal'}
0.690104 (0.023939) with: {'model__init_mode': 'glorot_uniform'}
0.647135 (0.057880) with: {'model__init_mode': 'he_normal'}
0.665365 (0.026557) with: {'model__init_mode': 'he_uniform'}
```

Output 14.6: Result of grid search on weight initialization

You can see that the best results were achieved with a uniform weight initialization scheme achieving a performance of about 72%.

14.8 How to Tune the Neuron Activation Function

The activation function controls the nonlinearity of individual neurons and when to fire. Generally, the rectifier activation function is the most popular. However, it used to be the sigmoid and the tanh functions, and these functions may still be more suitable for different problems.

In this example, you will evaluate the suite of different activation functions available in Keras. You will only use these functions in the hidden layer, as a sigmoid activation function is required in the output for the binary classification problem. Similar to the previous example, this is an argument to the `create_model()` function, and you will use the `model__` prefix for the `GridSearchCV` parameter grid. Generally, it is a good idea to prepare data to the range of the different transfer functions, which you will not do in this case.

The full code listing is provided below:

```
# Use scikit-learn to grid search the activation function
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model(activation='relu'):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), kernel_initializer='uniform',
                    activation=activation))
    model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
```

```
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
activation = ['softmax', 'softplus', 'softsign', 'relu', 'tanh', 'sigmoid',
              'hard_sigmoid', 'linear']
param_grid = dict(model__activation=activation)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.11: Grid search on activation function

Running this example produces the following output.

```
Best: 0.710938 using {'model__activation': 'linear'}
0.651042 (0.001841) with: {'model__activation': 'softmax'}
0.703125 (0.012758) with: {'model__activation': 'softplus'}
0.671875 (0.009568) with: {'model__activation': 'softsign'}
0.710938 (0.024080) with: {'model__activation': 'relu'}
0.669271 (0.019225) with: {'model__activation': 'tanh'}
0.675781 (0.011049) with: {'model__activation': 'sigmoid'}
0.677083 (0.004872) with: {'model__activation': 'hard_sigmoid'}
0.710938 (0.034499) with: {'model__activation': 'linear'}
```

Output 14.7: Result of grid search on activation function

Surprisingly (to me at least), the `linear` activation function achieved the best results with an accuracy of about 71%.

14.9 How to Tune Dropout Regularization

In this example, you will look at tuning the dropout rate for regularization in an effort to limit overfitting and improve the model's ability to generalize. For the best results, dropout is best combined with a weight constraint such as the max norm constraint. This involves fitting both the dropout percentage and the weight constraint. We will try dropout percentages between 0.0 and 0.9 (1.0 does not make sense) and MaxNorm weight constraint values between 0 and 5. The full code listing is provided below.

```
# Use scikit-learn to grid search the dropout rate
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.constraints import MaxNorm
from scikeras.wrappers import KerasClassifier
# Function to create model, required for KerasClassifier
def create_model(dropout_rate, weight_constraint):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), kernel_initializer='uniform',
                    activation='linear', kernel_constraint=MaxNorm(weight_constraint)))
    model.add(Dropout(dropout_rate))
    model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# fix random seed for reproducibility
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
print(dataset.dtype, dataset.shape)
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
weight_constraint = [1.0, 2.0, 3.0, 4.0, 5.0]
dropout_rate = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
param_grid = dict(model__dropout_rate=dropout_rate,
                  model__weight_constraint=weight_constraint)
#param_grid = dict(model__dropout_rate=dropout_rate)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.12: Grid search on dropout rate

Running this example produces the following output.

```
Best: 0.766927 using {'model__dropout_rate': 0.2, 'model__weight_constraint': 3.0}
0.729167 (0.021710) with: {'model__dropout_rate': 0.0, 'model__weight_constraint': 1.0}
0.746094 (0.022326) with: {'model__dropout_rate': 0.0, 'model__weight_constraint': 2.0}
0.753906 (0.022097) with: {'model__dropout_rate': 0.0, 'model__weight_constraint': 3.0}
0.750000 (0.012758) with: {'model__dropout_rate': 0.0, 'model__weight_constraint': 4.0}
0.751302 (0.012890) with: {'model__dropout_rate': 0.0, 'model__weight_constraint': 5.0}
...
```

Output 14.8: Result of grid search on dropout rate

You can see that the dropout rate of 20% and the MaxNorm weight constraint of 3 resulted in the best accuracy of about 77%. You may notice some of the result is `nan`. Probably it is due to the issue that the input is not normalized and you may run into a degenerated model by chance.

14.10 How to Tune the Number of Neurons in the Hidden Layer

The number of neurons in a layer is an important parameter to tune. Generally the number of neurons in a layer controls the representational capacity of the network, at least at that point in the topology. Generally, a large enough single layer network can approximate any other neural network, due to the universal approximation theorem.

In this example, you will look at tuning the number of neurons in a single hidden layer. you will try values from 1 to 30 in steps of 5. A larger network requires more training and at least the batch size and number of epochs should ideally be optimized with the number of neurons.

The full code listing is provided below.

```
# Use scikit-learn to grid search the number of neurons
import numpy as np
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from scikeras.wrappers import KerasClassifier
from tensorflow.keras.constraints import MaxNorm
# Function to create model, required for KerasClassifier
def create_model(neurons):
    # create model
    model = Sequential()
    model.add(Dense(neurons, input_shape=(8,), kernel_initializer='uniform',
                    activation='linear', kernel_constraint=MaxNorm(4)))
    model.add(Dropout(0.2))
    model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# fix random seed for reproducibility
```

```
seed = 7
tf.random.set_seed(seed)
# load dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = KerasClassifier(model=create_model, epochs=100, batch_size=10, verbose=0)
# define the grid search parameters
neurons = [1, 5, 10, 15, 20, 25, 30]
param_grid = dict(model__neurons=neurons)
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 14.13: Grid search on number of neurons in hidden layer

Running this example produces the following output.

```
Best: 0.729167 using {'model__neurons': 30}
0.701823 (0.010253) with: {'model__neurons': 1}
0.717448 (0.011201) with: {'model__neurons': 5}
0.717448 (0.008027) with: {'model__neurons': 10}
0.720052 (0.019488) with: {'model__neurons': 15}
0.709635 (0.004872) with: {'model__neurons': 20}
0.708333 (0.003683) with: {'model__neurons': 25}
0.729167 (0.009744) with: {'model__neurons': 30}
```

Output 14.9: Grid search on number of neurons in hidden layer

You can see that the best results were achieved with a network with 30 neurons in the hidden layer with an accuracy of about 73%.

14.11 Tips for Hyperparameter Optimization

This section lists some handy tips to consider when tuning hyperparameters of your neural network.

- ▷ **k-Fold Cross-Validation**. You can see that the results from the examples in this chapter show some variance. A default cross-validation of 3 was used, but perhaps $k = 5$ or $k = 10$ would be more stable. Carefully choose your cross-validation configuration to ensure your results are stable.
- ▷ **Review the Whole Grid**. Do not just focus on the best result, review the whole grid of results and look for trends to support configuration decisions.

- ▷ **Parallelize**. Use all your cores if you can, neural networks are slow to train and we often want to try a lot of different parameters. Consider spinning up a lot of AWS instances.
- ▷ **Use a Sample of Your Dataset**. Because networks are slow to train, try training them on a smaller sample of your training dataset, just to get an idea of general directions of parameters rather than optimal configurations.
- ▷ **Start with Coarse Grids**. Start with coarse-grained grids and zoom into finer grained grids once you can narrow the scope.
- ▷ **Do Not Transfer Results**. Results are generally problem specific. Try to avoid favorite configurations on each new problem that you see. It is unlikely that optimal results you discover on one problem will transfer to your next project. Instead look for broader trends like number of layers or relationships between parameters.
- ▷ **Reproducibility is a Problem**. Although we set the seed for the random number generator in NumPy, the results are not 100% reproducible. There is more to reproducibility when grid searching wrapped Keras models than is presented in this chapter.

14.12 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

Optimizers. Keras API Reference.
https://keras.io/api/optimizers/

Layer Weight Initializers. Keras API Reference.
https://keras.io/api/layers/initializers/

SciKeras documentation.
https://www.adriangb.com/scikeras/stable/

Routed parameters. SciKeras documentation.
https://www.adriangb.com/scikeras/stable/advanced.html#routed-parameters

Stratified K-Folds cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

Grid search cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Universal approximation theorem. Wikipedia.
https://en.wikipedia.org/wiki/Universal_approximation_theorem

14.13 Summary

In this chapter, you discovered how you can tune the hyperparameters of your deep learning networks in Python using Keras and scikit-learn. Specifically, you learned:

- ▷ How to wrap Keras models for use in scikit-learn and how to use grid search.
- ▷ How to grid search a suite of different standard neural network parameters for Keras models.
- ▷ How to design your own hyperparameter optimization experiments.

In the next chapter, you will learn how to preserve a Keras model after training.

Save and Load Your Keras Model with Serialization

15

Since deep learning models can take hours, days, and even weeks to train, it is important to know how to save and load them from disk. In this chapter, you will discover how to save your Keras models to files and load them up again to make predictions. After reading this chapter, you will know:

- ▷ How to save model weights and model architecture in separate files
- ▷ How to save model architecture in both YAML and JSON format
- ▷ How to save model weights and architecture into a single file for later use

Let's get started.

Overview

Keras separates the concerns of saving your model architecture and saving your model weights. Model weights are saved to HDF5 format. This grid format is ideal for storing multi-dimensional arrays of numbers. The model structure can be described and saved using two different formats: JSON and YAML.

In this chapter, you will look at three examples of saving and loading your model to a file:

- ▷ Save Model to JSON
- ▷ Save Model to YAML
- ▷ Save Model to HDF5

The examples will use the same simple network trained on the Pima Indians onset of diabetes binary classification dataset. (See Section 7.1)

Note: Saving models requires that you have the h5py library installed. It is usually installed as a dependency with TensorFlow. You can also install it easily as running:

```
sudo pip install h5py
```

15.1 Save Your Neural Network Model to JSON

JSON is a simple file format for describing data hierarchically. Keras provides the ability to describe any model using JSON format with a `to_json()` function. This can be saved to file and later loaded via the `model_from_json()` function that will create a new model from the JSON specification.

The weights are saved directly from the model using the `save_weights()` function and later loaded using the symmetrical `load_weights()` function. The example below trains and evaluates a simple model on the Pima Indians dataset. The model structure is then converted to JSON format and written to `model.json` in the local directory. The network weights are written to `model.h5` in the local directory.

The model and weight data is loaded from the saved files, and a new model is created. It is important to compile the loaded model before it is used. This is so that predictions made using the model can use the appropriate efficient computation from the Keras backend. The model is evaluated in the same way, printing the same evaluation score.

```
# MLP for Pima Indians Dataset Serialize to JSON and HDF5
from tensorflow.keras.models import Sequential, model_from_json
from tensorflow.keras.layers import Dense
import numpy
import os
# fix random seed for reproducibility
numpy.random.seed(7)
# load pima indians dataset
dataset = numpy.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model
model.fit(X, Y, epochs=150, batch_size=10, verbose=0)
# evaluate the model
scores = model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))

# serialize model to JSON
model_json = model.to_json()
with open("model.json", "w") as json_file:
    json_file.write(model_json)
# serialize weights to HDF5
model.save_weights("model.h5")
print("Saved model to disk")

# later...
```

```
# load json and create model
json_file = open('model.json', 'r')
loaded_model_json = json_file.read()
json_file.close()
loaded_model = model_from_json(loaded_model_json)
# load weights into new model
loaded_model.load_weights("model.h5")
print("Loaded model from disk")

# evaluate loaded model on test data
loaded_model.compile(loss='binary_crossentropy', optimizer='rmsprop',
                     metrics=['accuracy'])
score = loaded_model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (loaded_model.metrics_names[1], score[1]*100))
```

Listing 15.1: Serialize model to JSON format

Running this example provides the output below. It shows first the accuracy of the trained model, the saving of the model to disk in JSON format, the loading of the model and finally the re-evaluation of the loaded model achieving the same accuracy.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
acc: 78.78%
Saved model to disk
Loaded model from disk
acc: 78.78%
```

Output 15.1: Sample output from serializing model to JSON ofrmat

The JSON format of the model looks like the following:

```
{
  "class_name": "Sequential",
  "config": {
    "name": "sequential",
    "layers": [
      {
        "class_name": "InputLayer",
        "config": {
          "batch_input_shape": [
            null,
            8
          ],
          "dtype": "float32",
          "sparse": false,
          "ragged": false,
          "name": "dense_input"
        }
```

```
      },
      ...
      {
        "class_name": "Dense",
        "config": {
          "name": "dense_2",
          "trainable": true,
          "dtype": "float32",
          "units": 1,
          "activation": "sigmoid",
          "use_bias": true,
          "kernel_initializer": {
            "class_name": "GlorotUniform",
            "config": {
              "seed": null
            }
          },
          "bias_initializer": {
            "class_name": "Zeros",
            "config": {}
          },
          "kernel_regularizer": null,
          "bias_regularizer": null,
          "activity_regularizer": null,
          "kernel_constraint": null,
          "bias_constraint": null
        }
      }
    ]
  },
  "keras_version": "2.9.0",
  "backend": "tensorflow"
}
```

Listing 15.2: Sample JSON model file

15.2 Save Your Neural Network Model to YAML

Note: This method only applies to TensorFlow 2.5 or earlier. If you run it in later versions of TensorFlow, you will see a RuntimeError with the message "Method `model.to_yaml()` has been removed due to security risk of arbitrary code execution. Please use `model.to_json()` instead."

This example is much the same as the above JSON example, except the YAML format is used for the model specification. This example assumes that you have PyYAML 5, which can be installed with:

```
sudo pip install PyYAML
```

Listing 15.3: Installing PyYAML module

In this example, the model is described using YAML, saved to file `model.yaml`, and later loaded into a new model via the `model_from_yaml()` function. Weights are handled the same way as above in the HDF5 format as `model.h5`.

```
# MLP for Pima Indians Dataset serialize to YAML and HDF5
from tensorflow.keras.models import Sequential, model_from_yaml
from tensorflow.keras.layers import Dense
import numpy
import os
# fix random seed for reproducibility
seed = 7
numpy.random.seed(seed)
# load pima indians dataset
dataset = numpy.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model
model.fit(X, Y, epochs=150, batch_size=10, verbose=0)
# evaluate the model
scores = model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))

# serialize model to YAML
model_yaml = model.to_yaml()
with open("model.yaml", "w") as yaml_file:
    yaml_file.write(model_yaml)
# serialize weights to HDF5
model.save_weights("model.h5")
print("Saved model to disk")

# later...

# load YAML and create model
yaml_file = open('model.yaml', 'r')
loaded_model_yaml = yaml_file.read()
yaml_file.close()
loaded_model = model_from_yaml(loaded_model_yaml)
# load weights into new model
loaded_model.load_weights("model.h5")
print("Loaded model from disk")

# evaluate loaded model on test data
loaded_model.compile(loss='binary_crossentropy', optimizer='rmsprop',
                     metrics=['accuracy'])
```

```
score = loaded_model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (loaded_model.metrics_names[1], score[1]*100))
```

Listing 15.4: Serialize model to YAML format

Running the example displays the following output.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
acc: 78.78%
Saved model to disk
Loaded model from disk
acc: 78.78%
```

Output 15.2: Sample output from serializing model to YAML format

The model described in YAML format looks like the following:

```
backend: tensorflow
class_name: Sequential
config:
  layers:
  - class_name: Dense
    config:
      activation: relu
      activity_regularizer: null
      batch_input_shape: !!python/tuple
      - null
      - 8
      bias_constraint: null
      bias_initializer:
        class_name: Zeros
        config: {}
      bias_regularizer: null
      dtype: float32
      kernel_constraint: null
      kernel_initializer:
        class_name: VarianceScaling
        config:
          distribution: uniform
          mode: fan_avg
          scale: 1.0
          seed: null
      kernel_regularizer: null
      name: dense_1
      trainable: true
      units: 12
      use_bias: true
...
  - class_name: Dense
```

```
    config:
      activation: sigmoid
      activity_regularizer: null
      bias_constraint: null
      bias_initializer:
        class_name: Zeros
        config: {}
      bias_regularizer: null
      dtype: float32
      kernel_constraint: null
      kernel_initializer:
        class_name: VarianceScaling
        config:
          distribution: uniform
          mode: fan_avg
          scale: 1.0
          seed: null
      kernel_regularizer: null
      name: dense_3
      trainable: true
      units: 1
      use_bias: true
  name: sequential_1
keras_version: 2.2.5
```

Listing 15.5: Sample YAML model file

15.3 Save Model Weights and Architecture Together

Keras also supports a simpler interface to save both the model weights and model architecture together into a single H5 file. Saving the model in this way includes everything you need to know about the model, including:

- ▷ Model weights
- ▷ Model architecture
- ▷ Model compilation details (loss and metrics)
- ▷ Model optimizer state

This means that you can load and use the model directly without having to re-compile it as you did to in the examples above. This is the preferred way for saving and loading your Keras model.

How to Save a Keras Model

You can save your model by calling the `save()` function on the model and specifying the filename. The example below demonstrates this by first fitting a model, evaluating it, and saving it to the file `model.h5`.

```
# MLP for Pima Indians Dataset saved to single file
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# load pima indians dataset
dataset = loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# define model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model
model.fit(X, Y, epochs=150, batch_size=10, verbose=0)
# evaluate the model
scores = model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
# save model and architecture to single file
model.save("model.h5")
print("Saved model to disk")
```

Listing 15.6: Example of saving weights and architecture to single file

Running the example fits the model, summarizes the model's performance on the training dataset, and saves the model to file.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
acc: 77.73%
Saved model to disk
```

Output 15.3: Sample output from saving weights and architecture to a single file

You can later load this model from the file and use it. Note that in the Keras library, there is another function doing the same, as follows:

```
...
# equivalent to: model.save("model.h5")
from tensorflow.keras.models import save_model
save_model(model, "model.h5")
```

Listing 15.7: Another function in Keras to save a model

How to Load a Keras Model

Your saved model can then be loaded later by calling the `load_model()` function and passing the filename. The function returns the model with the same architecture and weights. In this case, you load the model, summarize the architecture, and evaluate it on the same dataset to confirm the weights and architecture are the same.

```
# load and evaluate a saved model
from numpy import loadtxt
from tensorflow.keras.models import load_model

# load model
model = load_model('model.h5')
# summarize model.
model.summary()
# load dataset
dataset = loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# evaluate the model
score = model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], score[1]*100))
```

Listing 15.8: Example of loading weights and architecture from single file

Running the example first loads the model, prints a summary of the model architecture, and then evaluates the loaded model on the same dataset. The model achieves the same accuracy score, which in this case is 77%.

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 12)                108
_________________________________________________________________
dense_2 (Dense)              (None, 8)                 104
_________________________________________________________________
dense_3 (Dense)              (None, 1)                 9
=================================================================
Total params: 221
Trainable params: 221
Non-trainable params: 0
_________________________________________________________________

acc: 77.73%
```

Output 15.4: Sample output from saving weights and architecture to a single file

Protocol Buffer Format

While saving and loading a Keras model using HDF5 format is the recommended way, TensorFlow supports yet another format, the *protocol buffer*. It is considered faster to save

and load a protocol buffer format but doing so will produce multiple files. The syntax is the same, except that you do not need to provide the `.h5` extension to the filename:

```
# save model and architecture to single file
model.save("model")

# ... later

# load model
model = load_model('model')
# print summary
model.summary()
```

Listing 15.9: Saving a Keras model in protocol buffer format

These will create a directory "`model`" with the following files:

```
model/
|-- assets/
|-- keras_metadata.pb
|-- saved_model.pb
`-- variables/
    |-- variables.data-00000-of-00001
    `-- variables.index
```

Output 15.5: Directory tree of the generated model files

This is also the format used to save a model in TensorFlow v1.x. You may encounter this when you download a pretrained model from TensorFlow Hub.

15.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

How can I save a Keras model? Keras FAQ.
https://keras.io/getting-started/faq/%5C#how-can-i-save-a-keras-model

The Models API. Keras API Reference.
https://keras.io/api/models/

TensorFlow Hub.
https://www.tensorflow.org/hub

Protocol buffers.
https://developers.google.com/protocol-buffers/

15.5 Summary

Saving and loading models is an important capability for transplanting a deep learning model from research and development to operations. In this chapter you discovered how to serialize your Keras deep learning models. You learned:

- How to save model weights to HDF5 formatted files and load them again later.
- How to save Keras model definitions to JSON files and load them again.
- How to save Keras model definitions to YAML files and load them again.

In the next chapter, we will see how we can make Keras save the model automatically during training.

Keep the Best Models During Training with Checkpointing

16

Deep learning models can take hours, days, or even weeks to train. If the run is stopped unexpectedly, you can lose a lot of work. In this chapter, you will discover how to checkpoint your deep learning models during training in Python using the Keras library. After completing this chapter, you will know:

- ▷ The importance of checkpointing neural network models when training.
- ▷ How to checkpoint each improvement to a model during training.
- ▷ How to checkpoint the very best model observed during training.

Let's get started.

Overview

This chapter is in five parts; they are:

- ▷ Checkpointing Neural Network Models
- ▷ Checkpoint Neural Network Model Improvements
- ▷ Checkpoint the Best Neural Network Model Only
- ▷ Use EarlyStopping Together with Checkpoint
- ▷ Loading a Saved Neural Network Model

16.1 Checkpointing Neural Network Models

Application checkpointing is a fault tolerance technique for long-running processes. In this approach, a snapshot of the state of the system is taken in case of system failure. If there is a problem, not all is lost. The checkpoint may be used directly or as the starting point for a new run, picking up where it left off. When training deep learning models, the checkpoint captures the weights of the model. These weights can be used to make predictions as-is or as the basis for ongoing training.

The Keras library provides a checkpointing capability by a callback API. The `ModelCheckpoint` callback class allows you to define where to checkpoint the model weights,

how to name the file, and under what circumstances to make a checkpoint of the model. The API allows you to specify which metric to monitor, such as loss or accuracy on the training or validation dataset. You can specify whether to look for an improvement in maximizing or minimizing the score. Finally, the filename you use to store the weights can include variables like the epoch number or metric. The `ModelCheckpoint` can then be passed to the training process when calling the `fit()` function on the model. Note that you may need to install the h5py library to output network weights in HDF5 format.

16.2 Checkpoint Neural Network Model Improvements

A good use of checkpointing is to output the model weights each time an improvement is observed during training. The example below creates a small neural network for the Pima Indians onset of diabetes binary classification problem (see Section 7.1). The example assumes that the `pima-indians-diabetes.csv` file is in your working directory. The example uses 33% of the data for validation.

Checkpointing is set up to save the network weights only when there is an improvement in classification accuracy on the validation dataset (`monitor='val_accuracy'` and `mode='max'`). The weights are stored in a file that includes the score in the filename, `weights-improvement-{val_accuracy:.2f}.hdf5`.

```
# Checkpoint the weights when validation accuracy improves
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
tf.random.set_seed(42)
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# checkpoint
filepath="weights-improvement-{epoch:02d}-{val_accuracy:.2f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_accuracy', verbose=1,
                             save_best_only=True, mode='max')
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10,
          callbacks=[checkpoint], verbose=0)
```

Listing 16.1: Checkpoint model improvements

Running the example produces the output below, truncated for brevity. In the output you can see cases where an improvement in the model accuracy on the validation dataset resulted in a new weight file being written to disk.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
...
Epoch 00139: val_accuracy did not improve
Epoch 00140: val_accuracy improved from 0.83465 to 0.83858, saving model to
↪  weights-improvement-140-0.84.hdf5
Epoch 00141: val_accuracy did not improve
Epoch 00142: val_accuracy did not improve
Epoch 00143: val_accuracy did not improve
Epoch 00144: val_accuracy did not improve
Epoch 00145: val_accuracy did not improve
Epoch 00146: val_accuracy improved from 0.83858 to 0.84252, saving model to
↪  weights-improvement-146-0.84.hdf5
Epoch 00147: val_accuracy did not improve
Epoch 00148: val_accuracy improved from 0.84252 to 0.84252, saving model to
↪  weights-improvement-148-0.84.hdf5
Epoch 00149: val_accuracy did not improve
```

Output 16.1: Sample output from checkpoint model improvements

You will see a number of files in your working directory containing the network weights in HDF5 format. For example:

```
...
weights-improvement-53-0.76.hdf5
weights-improvement-71-0.76.hdf5
weights-improvement-77-0.78.hdf5
weights-improvement-99-0.78.hdf5
```

This is a very simple checkpointing strategy. It may create a lot of unnecessary checkpoint files if the validation accuracy moves up and down over training epochs. Nevertheless, it will ensure you have a snapshot of the best model discovered during your run.

16.3 Checkpoint the Best Neural Network Model Only

A simpler checkpoint strategy is to save the model weights to the same file if and only if the validation accuracy improves. This can be done easily using the same code from above and changing the output filename to be fixed (not including score or epoch information). In this case, model weights are written to the file `weights.best.hdf5` only if the classification accuracy of the model on the validation dataset improves over the best seen so far.

```
# Checkpoint the weights for best model on validation accuracy
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import numpy as np
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# checkpoint
filepath="weights.best.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_accuracy', verbose=1,
                             save_best_only=True, mode='max')
callbacks_list = [checkpoint]
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10,
          callbacks=callbacks_list, verbose=0)
```

Listing 16.2: Checkpoint best model only

Running this example provides the following output (truncated for brevity):

```
...
Epoch 00139: val_accuracy improved from 0.79134 to 0.79134, saving model to weights.best.hdf5
Epoch 00140: val_accuracy did not improve
Epoch 00141: val_accuracy did not improve
Epoch 00142: val_accuracy did not improve
Epoch 00143: val_accuracy did not improve
Epoch 00144: val_accuracy improved from 0.79134 to 0.79528, saving model to weights.best.hdf5
Epoch 00145: val_accuracy improved from 0.79528 to 0.79528, saving model to weights.best.hdf5
Epoch 00146: val_accuracy did not improve
Epoch 00147: val_accuracy did not improve
Epoch 00148: val_accuracy did not improve
Epoch 00149: val_accuracy did not improve
```

Output 16.2: Sample output from checkpoint the best model

You should see the weight file, `weights.best.hdf5`, in your local directory. This is a handy checkpoint strategy to always use during your experiments.

It will ensure that your best model is saved for the run for you to use later if you wish. It avoids needing to include any code to manually keep track and serialize the best model when training.

16.4 Use EarlyStopping Together with Checkpoint

In the examples above, an attempt was made to fit your model with 150 epochs. In reality, it is not easy to tell how many epochs you need to train your model. One way to address this problem is to overestimate the number of epochs. But this may take significant time. After all, if you are checkpointing the best model only, you may find that over the several thousand epochs run, we already achieved the best model in the first hundred epochs, and no more checkpoints are made afterward.

It is quite common to use the `ModelCheckpoint` callback together with `EarlyStopping`. It helps to stop the training once no metric improvement is seen for several epochs. The example below adds the callback `es` to make the training stop early once it does not see the validation accuracy improve for five consecutive epochs:

```
# Checkpoint the weights for best model on validation accuracy
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import matplotlib.pyplot as plt
import numpy as np
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# checkpoint
filepath="weights.best.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_accuracy', verbose=1,
                             save_best_only=True, mode='max')
es = EarlyStopping(monitor='val_accuracy', patience=5)
callbacks_list = [checkpoint, es]
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10,
          callbacks=callbacks_list, verbose=0)
```

Listing 16.3: Checkpoint best model with early stopping

Running this example provides the following output:

```
Epoch 1: val_accuracy improved from -inf to 0.51969, saving model to weights.best.hdf5
Epoch 2: val_accuracy did not improve from 0.51969
Epoch 3: val_accuracy improved from 0.51969 to 0.54724, saving model to weights.best.hdf5
Epoch 4: val_accuracy improved from 0.54724 to 0.61417, saving model to weights.best.hdf5
Epoch 5: val_accuracy did not improve from 0.61417
Epoch 6: val_accuracy did not improve from 0.61417
```

```
Epoch 7: val_accuracy improved from 0.61417 to 0.66142, saving model to weights.best.hdf5
Epoch 8: val_accuracy did not improve from 0.66142
Epoch 9: val_accuracy did not improve from 0.66142
Epoch 10: val_accuracy improved from 0.66142 to 0.68504, saving model to weights.best.hdf5
Epoch 11: val_accuracy did not improve from 0.68504
Epoch 12: val_accuracy did not improve from 0.68504
Epoch 13: val_accuracy did not improve from 0.68504
Epoch 14: val_accuracy did not improve from 0.68504
Epoch 15: val_accuracy improved from 0.68504 to 0.69685, saving model to weights.best.hdf5
Epoch 16: val_accuracy improved from 0.69685 to 0.71260, saving model to weights.best.hdf5
Epoch 17: val_accuracy improved from 0.71260 to 0.72047, saving model to weights.best.hdf5
Epoch 18: val_accuracy did not improve from 0.72047
Epoch 19: val_accuracy did not improve from 0.72047
Epoch 20: val_accuracy did not improve from 0.72047
Epoch 21: val_accuracy did not improve from 0.72047
Epoch 22: val_accuracy did not improve from 0.72047
```

Output 16.3: Sample output from checkpoint the best model with early stopping

This training process stopped after epoch 22 as there are no better accuracy achieved for the last 5 epochs.

16.5 Loading a Saved Neural Network Model

Now that you have seen how to checkpoint your deep learning models during training, you need to review how to load and use a checkpointed model. The checkpoint only includes the model weights. It assumes you know the network structure. This, too, can be serialized to a file in JSON or YAML format. In the example below, the model structure is known, and the best weights are loaded from the previous experiment, stored in the working directory in the `weights.best.hdf5` file. The model is then used to make predictions on the entire dataset.

```
# How to load and use weights from a checkpoint
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import numpy as np
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# load weights
model.load_weights("weights.best.hdf5")
# Compile model (required to make predictions)
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print("Created model and loaded weights from file")
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
```

```
Y = dataset[:,8]
# estimate accuracy on whole dataset using loaded weights
scores = model.evaluate(X, Y, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
```

Listing 16.4: Load and evaluate a model checkpoint

Running the example produces the following output:

```
Created model and loaded weights from file
acc: 77.73%
```

Output 16.4: Sample output from loading and evaluating a model checkpoint

16.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

ModelCheckpoint. Keras API Reference.
https://keras.io/api/callbacks/model_checkpoint/

Callbacks API. Keras API Reference.
https://keras.io/api/callbacks/

16.7 Summary

In this chapter, you discovered the importance of checkpointing deep learning models for long training runs. You learned:

- ▷ How to use Keras to checkpoint each time an improvement to the model is observed.
- ▷ How to only checkpoint the very best model observed during training.
- ▷ How to load a checkpointed model from file and use it later to make predictions.

In the next chapter, you will learn how to visualize the training progress.

Understand Model Behavior During Training by Plotting History

17

You can learn a lot about neural networks and deep learning models by observing their performance over time during training. For example, if you see the training accuracy went worse with training epochs, you know you have issue with the optimization. Probably your learning rate is too fast. In this chapter, you will discover how you can review and visualize the performance of deep learning models over time during training in Python with Keras. After completing this chapter, you will know:

- ▷ How to inspect the history metrics collected during training
- ▷ How to plot accuracy metrics on training and validation datasets during training
- ▷ How to plot model loss metrics on training and validation datasets during training

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Access Model Training History in Keras
- ▷ Visualize Model Training History in Keras

17.1 Access Model Training History in Keras

Keras provides the capability to register callbacks when training a deep learning model. One of the default callbacks registered when training all deep learning models is the History callback. It records training metrics for each epoch. This includes the loss and the accuracy (for classification problems) and the loss and accuracy for the validation dataset if one is set.

The history object is returned from calls to the `fit()` function used to train the model. Metrics are stored in a dictionary in the `history` member of the object returned. For example, you can list the metrics collected in a history object using the following snippet of code after a model is trained:

```
...
# list all data in history
print(history.history.keys())
```

Listing 17.1: Display recorded history metric names

For example, for a model trained on a classification problem with a validation dataset, this might produce the following listing:

```
['accuracy', 'loss', 'val_accuracy', 'val_loss']
```

Output 17.1: Sample output from recorded history metric names

Usually, when you called `compile()` with your Keras model, you provided the loss function as well as other metrics. The loss function is the objective to optimize. Training a neural network is to minimize the loss. The metrics are just for reporting but usually useful for the user of the network. In case of classification, "accuracy" is commonly used as metric while cross-entropy is used as the loss function. You will learn more about the loss function in Chapter 19.

You can use the data collected in the history object to create plots. The plots can provide an indication of useful things about the training of the model, such as:

- ▷ Its speed of convergence over epochs (slope)
- ▷ Whether the model may have already converged (plateau of the line)
- ▷ Whether the model may be over-learning the training data (inflection for validation line)

17.2 Visualize Model Training History in Keras

You can create plots from the collected history data. In the example below, a small network to model the Pima Indians onset of diabetes binary classification problem (see Section 7.1).

The example collects the history returned from training the model and creates two charts:

1. A plot of accuracy on the training and validation datasets over training epochs
2. A plot of loss on the training and validation datasets over training epochs

```
# Visualize training history
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
```

```
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model
history = model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10, verbose=0)
# list all data in history
print(history.history.keys())
# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
# summarize history for loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

Listing 17.2: Evaluate a model and plot training history

The plots are provided below. The history for the validation dataset is labeled test by convention as it is indeed a test dataset for the model. From the plot of the accuracy, you can see that the model could probably be trained a little more as the trend for accuracy on both datasets is still rising for the last few epochs. You can also see that the model has not yet over-learned the training dataset, showing comparable skill on both datasets.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

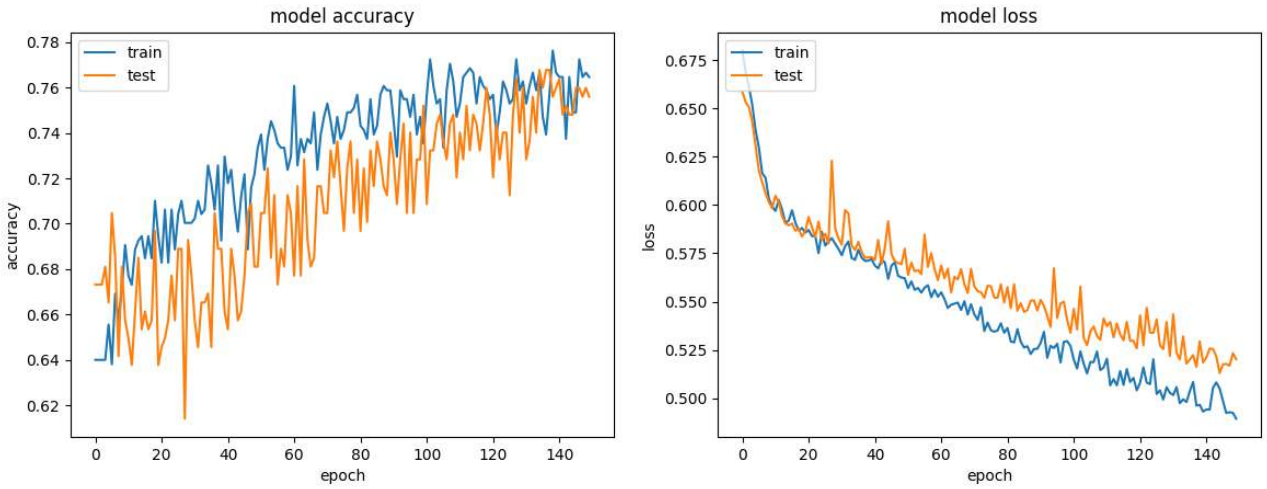

Figure 17.1: Plot of model accuracy (left) and loss (right) on training and validation datasets

From the plot of the loss, you can see that the model has comparable performance on both train and validation datasets (labeled test). If these parallel plots start to depart consistently, it might be a sign to stop training at an earlier epoch.

17.3 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

tf.keras.callbacks.History. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/History

17.4 Summary

In this chapter, you discovered the importance of collecting and reviewing metrics while training your deep learning models. You learned:

- ▷ How to inspect a history object returned from training to discover the metrics that were collected.
- ▷ How to extract model accuracy information for training and validation datasets and plot the data.
- ▷ How to extract and plot the model loss information calculated from training and validation datasets.

In the next chapter, you will learn about the different activation functions available in Keras.

Using Activation Functions in Neural Networks

18

Activation functions play an integral role in neural networks by introducing nonlinearity. This nonlinearity allows neural networks to develop complex representations and functions based on the inputs that would not be possible with a simple linear regression model.

Many different nonlinear activation functions have been proposed throughout the history of neural networks. In this chapter, you will explore three popular ones: sigmoid, tanh, and ReLU.

After reading this chapter, you will learn:

- ▷ Why nonlinearity is important in a neural network
- ▷ How different activation functions can contribute to the vanishing gradient problem
- ▷ Sigmoid, tanh, and ReLU activation functions
- ▷ How to use different activation functions in your TensorFlow model

Let's get started.

Overview

This chapter is divided into five parts; they are:

- ▷ Why do we need nonlinear activation functions
- ▷ Sigmoid function and vanishing gradient
- ▷ Hyperbolic tangent function
- ▷ Rectified Linear Unit (ReLU)
- ▷ Using the activation functions in practice

18.1 Why Do We Need Nonlinear Activation Functions

You might be wondering, why all this hype about nonlinear activation functions? Or why can't we just use an identity function after the weighted linear combination of activations from the previous layer? Using multiple linear layers is basically the same as using a single

linear layer. This can be seen through a simple example. Let's say you have a one hidden layer neural network, each with two hidden neurons.

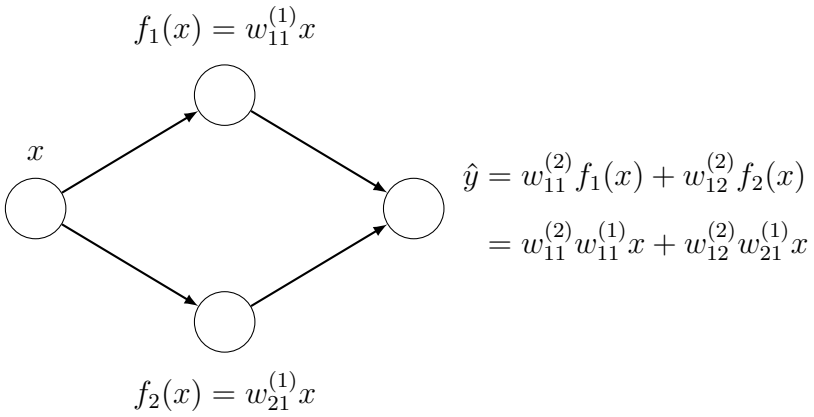

Figure 18.1: Single hidden layer neural network with linear layers

You can then rewrite the output layer as a linear combination of the original input variable if you used a linear hidden layer. If you had more neurons and weights, the equation would be a lot longer with more nesting and more multiplications between successive layer weights. However, the idea remains the same: You can represent the entire network as a single linear layer. To make the network represent more complex functions, you would need nonlinear activation functions. Let's start with a popular example, the sigmoid function.

18.2 Sigmoid Function and Vanishing Gradient

The sigmoid activation function is a popular choice for the nonlinear activation function for neural networks. One reason it's popular is that it has output values between 0 and 1, which mimic probability values. Hence it is used to convert the real-valued output of a linear layer to a probability, which can be used as a probability output. This also makes it an important part of logistic regression methods, which can be used directly for binary classification.

The sigmoid function is commonly represented by σ and has the form $\sigma = \frac{1}{1+e^{-x}}$. In TensorFlow, you can call the sigmoid function from the Keras library as follows:

```
import tensorflow as tf
from tensorflow.keras.activations import sigmoid

input_array = tf.constant([-1, 0, 1], dtype=tf.float32)
print(sigmoid(input_array))
```

Listing 18.1: Calling a sigmoidal function

This gives the following output:

```
tf.Tensor([0.26894143 0.5        0.7310586 ], shape=(3,), dtype=float32)
```

Output 18.1: Output of a sigmoidal function

You can also plot the sigmoid function as a function of x. When looking at the activation function for the neurons in a neural network, you should also be interested in its derivative due to backpropagation and the chain rule, which would affect how the neural network learns from data.

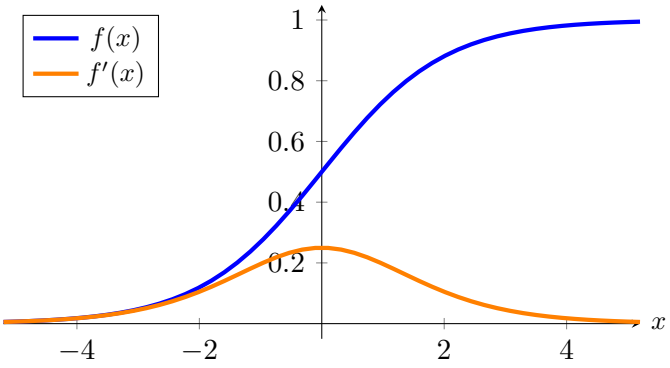

Figure 18.2: Sigmoid activation function (blue) and gradient (orange)

Here, you can observe that the gradient of the sigmoid function is always between 0 and 0.25. And as the x tends to positive or negative infinity, the gradient tends to zero. This could contribute to the vanishing gradient problem, meaning when the inputs are at some large magnitude of x (e.g., due to the output from earlier layers), the gradient is too small to initiate the correction.

Vanishing gradient is a problem because the chain rule is used in backpropagation in deep neural networks. Recall that in neural networks, the gradient (of the loss function) at each layer is the gradient at its subsequent layer multiplied by the gradient of its activation function. As there are many layers in the network, if the gradient of the activation functions is less than 1, the gradient at some layer far away from the output will be close to zero. And any layer with a gradient close to zero will stop the gradient propagation further back to the earlier layers.

Since the sigmoid function is always less than 1, a network with more layers would exacerbate the vanishing gradient problem. Furthermore, there is a saturation region where the gradient of the sigmoid tends to 0, which is where the magnitude of x is large. So, if the output of the weighted sum of activations from previous layers is large, then you would have a very small gradient propagating through this neuron, as the derivative of the activation a with respect to the input to the activation function would be small (in the saturation region).

Granted, there is also the derivative of the linear term with respect to the previous layer's activations which might be greater than 1 for the layer since the weight might be large, and it's a sum of derivatives from the different neurons. However, it might still raise concern at the start of training as weights are usually initialized to be small.

18.3 Hyperbolic Tangent Function

Another activation function to consider is the tanh activation function, also known as the hyperbolic tangent function. It has a larger range of output values compared to the sigmoid

function and a larger maximum gradient. The tanh function is a hyperbolic analog to the normal tangent function for circles that most people are familiar with.

Plotting out the tanh function and its gradient:

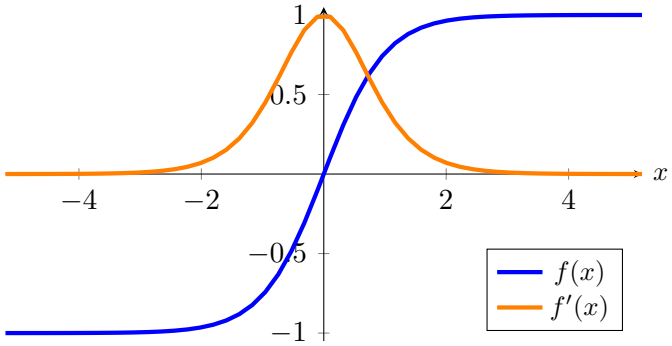

Figure 18.3: Tanh activation function (blue) and gradient (orange)

Notice that the gradient now has a maximum value of 1, compared to the sigmoid function, where the largest gradient value is 0.25. This makes a network with tanh activation less susceptible to the vanishing gradient problem. However, the tanh function also has a saturation region, where the value of the gradient tends toward as the magnitude of the input x gets larger.

In TensorFlow, you can implement the tanh activation on a tensor using the `tanh` function in Keras' activations module:

```
import tensorflow as tf
from tensorflow.keras.activations import tanh

input_array = tf.constant([-1, 0, 1], dtype=tf.float32)
print(tanh(input_array))
```

Listing 18.2: Calling a hyperbolic tangent function

This gives the output:

```
tf.Tensor([-0.7615942  0.         0.7615942], shape=(3,), dtype=float32)
```

Output 18.2: Output of the tanh function

18.4 Rectified Linear Unit (ReLU)

The last activation function to cover in detail is the Rectified Linear Unit, also popularly known as ReLU. It has become popular recently due to its relatively simple computation. This helps to speed up neural networks and seems to get empirically good performance, which makes it a good starting choice for the activation function.

The ReLU function is a simple $\max(0, x)$ function, which can also be thought of as a piecewise function with all inputs less than 0 mapping to 0 and all inputs greater than or equal to 0 mapping back to themselves (i.e., identity function). Graphically,

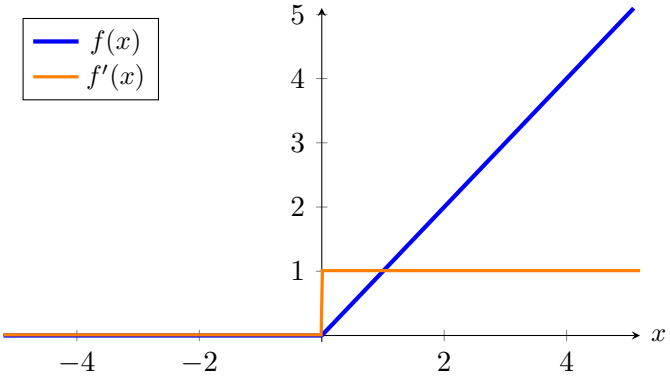

Figure 18.4: ReLU activation function (blue) and gradient (orange)

Notice that the gradient of ReLU is 1 whenever the input is positive, which helps address the vanishing gradient problem. However, whenever the input is negative, the gradient is 0. This can cause another problem, the dead neuron/dying ReLU problem, which is an issue if a neuron is *persistently inactivated*. In this case, the neuron can never learn, and its weights are never updated due to the chain rule as it has a 0 gradient as one of its terms. If this happens for all data in your dataset, then it can be very difficult for this neuron to learn from your dataset unless the activations in the previous layer change such that the neuron is no longer "dead".

To use the ReLU activation in TensorFlow:

```
import tensorflow as tf
from tensorflow.keras.activations import relu

input_array = tf.constant([-1, 0, 1], dtype=tf.float32)
print(relu(input_array))
```

Listing 18.3: Calling a rectified linear unit function

This gives the following output:

```
tf.Tensor([0. 0. 1.], shape=(3,), dtype=float32)
```

Output 18.3: Output of the ReLU function

The three activation functions reviewed above show that they are all monotonically increasing functions. This is required; otherwise, you cannot apply the gradient descent algorithm.

Now that you've explored some common activation functions and how to use them in TensorFlow. Let's take a look at how you can use these in practice in an actual model.

18.5 Using Activation Functions in Practice

Before exploring the use of activation functions in practice, let's look at another common way to use activation functions when combining them with another Keras layer. Let's say you want to add a ReLU activation on top of a `Dense` layer. One way you can do this following the above methods shown is to do:

```
x = Dense(units=10)(input_layer)
x = relu(x)
```

Listing 18.4: Connecting an activation function to a Keras layer

However, for many Keras layers, you can also use a more compact representation to add the activation on top of the layer:

```
x = Dense(units=10, activation="relu")(input_layer)
```

Listing 18.5: Setting activation function to a Keras layer

Using this more compact representation, let's build our LeNet5 model using Keras:

```
import tensorflow as tf
import tensorflow.keras as keras
from tensorflow.keras.layers import Dense, Input, Flatten, \
                                    Conv2D, BatchNormalization, MaxPool2D
from tensorflow.keras.models import Model

(trainX, trainY), (testX, testY) = keras.datasets.cifar10.load_data()

input_layer = Input(shape=(32,32,3,))
x = Conv2D(6, (5,5), padding="same", activation="relu")(input_layer)
x = MaxPool2D(pool_size=(2,2))(x)
x = Conv2D(16, (5,5), padding="same", activation="relu")(x)
x = MaxPool2D(pool_size=(2, 2))(x)
x = Conv2D(120, (5,5), padding="same", activation="relu")(x)
x = Flatten()(x)
x = Dense(units=84, activation="relu")(x)
x = Dense(units=10, activation="softmax")(x)

model = Model(inputs=input_layer, outputs=x)
model.summary()

model.compile(optimizer="adam", loss=tf.keras.losses.SparseCategoricalCrossentropy(),
              metrics="acc")

history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
                    validation_data=(testX, testY))
```

Listing 18.6: Building the LeNet5 network and train with CIFAR-10 dataset

And running this code gives the following output:

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 32, 32, 3)]       0

 conv2d (Conv2D)             (None, 32, 32, 6)         456

 max_pooling2d (MaxPooling2D)  (None, 16, 16, 6)       0
```

```
 conv2d_1 (Conv2D)              (None, 16, 16, 16)          2416

 max_pooling2d_1 (MaxPooling2D)  (None, 8, 8, 16)           0

 conv2d_2 (Conv2D)              (None, 8, 8, 120)           48120

 flatten (Flatten)              (None, 7680)                0

 dense (Dense)                  (None, 84)                  645204

 dense_1 (Dense)                (None, 10)                  850

=================================================================
Total params: 697,046
Trainable params: 697,046
Non-trainable params: 0
_________________________________________________________________
Epoch 1/10
196/196 [==============================] - 14s 11ms/step - loss: 2.9758 acc: 0.3390 - val_loss: 1.55
Epoch 2/10
196/196 [==============================] - 2s 8ms/step - loss: 1.4319 - acc: 0.4927 - val_loss: 1.38
Epoch 3/10
196/196 [==============================] - 2s 8ms/step - loss: 1.2505 - acc: 0.5583 - val_loss: 1.35
Epoch 4/10
196/196 [==============================] - 2s 8ms/step - loss: 1.1127 - acc: 0.6094 - val_loss: 1.28
Epoch 5/10
196/196 [==============================] - 2s 8ms/step - loss: 0.9763 - acc: 0.6594 - val_loss: 1.32
Epoch 6/10
196/196 [==============================] - 2s 8ms/step - loss: 0.8510 - acc: 0.7017 - val_loss: 1.39
Epoch 7/10
196/196 [==============================] - 2s 8ms/step - loss: 0.7361 - acc: 0.7426 - val_loss: 1.41
Epoch 8/10
196/196 [==============================] - 2s 8ms/step - loss: 0.6060 - acc: 0.7894 - val_loss: 1.53
Epoch 9/10
196/196 [==============================] - 2s 8ms/step - loss: 0.5020 - acc: 0.8265 - val_loss: 1.78
Epoch 10/10
196/196 [==============================] - 2s 8ms/step - loss: 0.4013 - acc: 0.8605 - val_loss: 1.83
```

Output 18.4: Output of training a LeNet5 network

And that's how you can use different activation functions in your TensorFlow models!

18.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

Layer activation functions. Keras API Reference.
https://keras.io/api/layers/activations/

tf.keras.layers.ReLU. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/layers/ReLU

tf.keras.layers.LeakyReLU. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/layers/LeakyReLU

Papers

Kaiming He et al. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of IEEE International Conference on Computer Vision (ICCV)*. 2015.
https://arxiv.org/abs/1502.01852

Ian J. Goodfellow et al. "Maxout Networks". In: *Proceedings of the 30th International Conference on Machine Learning*. Atlanta, GA, USA, 2013.
https://arxiv.org/abs/1302.4389

18.7 Summary

In this chapter, you have seen why activation functions are important to allow for the complex neural networks that are common in deep learning today. You have also seen some popular activation functions, their derivatives, and how to integrate them into your TensorFlow models. Specifically, you learned:

- ▷ Why nonlinearity is important in a neural network
- ▷ How different activation functions can contribute to the vanishing gradient problem
- ▷ Sigmoid, tanh, and ReLU activation functions
- ▷ How to use different activation functions in your TensorFlow model

In the next chapter, you will learn about the various loss functions.

Loss Functions in TensorFlow

19

The loss metric is very important for neural networks. As all machine learning models are one optimization problem or another, the loss is the objective function to minimize. In neural networks, the optimization is done with gradient descent and backpropagation. But what are loss functions, and how are they affecting your neural networks?

In this chapter, you will learn what loss functions are and delve into some commonly used loss functions and how you can apply them to your neural networks. After reading this chapter, you will learn:

- ▷ What are loss functions, and how they are different from metrics
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your TensorFlow model

Let's get started!

Overview

This chapter is divided into five sections; they are:

- ▷ What Are Loss Functions?
- ▷ Mean Absolute Error
- ▷ Mean Squared Error
- ▷ Categorical Cross-Entropy
- ▷ Loss Functions in Practice

19.1 What Are Loss Functions?

In neural networks, loss functions help optimize the performance of the model. They are usually used to measure some penalty that the model incurs on its predictions, such as the deviation of the prediction away from the ground truth label. Loss functions are usually differentiable across their domain (but it is allowed that the gradient is undefined only for very specific points, such as $x = 0$, which is basically ignored in practice). In the training

loop, they are differentiated with respect to parameters, and these gradients are used for your backpropagation and gradient descent steps to optimize your model on the training set.

Loss functions are also slightly different from metrics. While loss functions can tell you the performance of our model, they might not be of direct interest or easily explainable by humans. This is where metrics come in. Metrics such as accuracy are much more useful for humans to understand the performance of a neural network even though they might not be good choices for loss functions since they might not be differentiable.

In the following, let's explore some common loss functions: the mean absolute error, mean squared error, and categorical cross-entropy.

19.2 Mean Absolute Error

The mean absolute error (MAE) measures the absolute difference between predicted values and the ground truth labels and takes the mean of the difference across all training examples. Mathematically, it is equal to $\frac{1}{m}\sum_{i=1}^{m}|\hat{y}_i - y_i|$ where m is the number of training examples y_i and $\hat{y}_i$ are the ground truth and predicted values, respectively, averaged over all training examples. The MAE is never negative and would be zero only if the prediction matched the ground truth perfectly. It is an intuitive loss function and might also be used as one of your metrics, specifically for regression problems, since you want to minimize the error in your predictions.

Let's look at what the mean absolute error loss function looks like graphically:

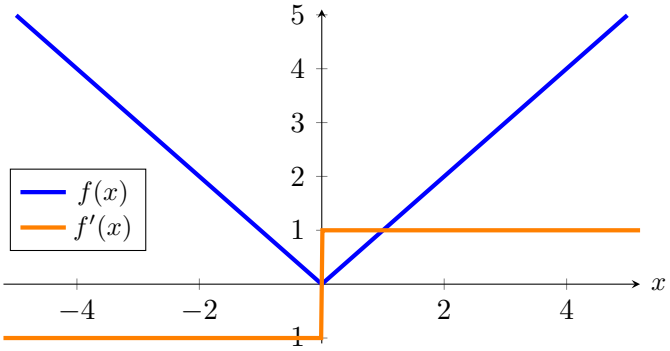

Figure 19.1: Mean absolute error loss function (blue) and gradient (orange)

Similar to activation functions, you might also be interested in what the gradient of the loss function looks like since you are using the gradient later to do backpropagation to train your model's parameters.

You might notice a discontinuity in the gradient function for the mean absolute loss function. Many tend to ignore it since it occurs only at $x = 0$, which, in practice, rarely happens since it is the probability of a single point in a continuous distribution.

Let's take a look at how to implement this loss function in TensorFlow using the Keras losses module:

```python
import tensorflow as tf
from tensorflow.keras.losses import MeanAbsoluteError

y_true = [1., 0.]
y_pred = [2., 3.]

mae_loss = MeanAbsoluteError()

print(mae_loss(y_true, y_pred).numpy())
```

Listing 19.1: Compute the mean absolute error

This gives you `2.0` as the output as expected, since $\frac{1}{2}(|2-1|+|3-0|) = \frac{1}{2}(4) = 4$. Next, let's explore another loss function for regression models with slightly different properties, the mean squared error.

19.3 Mean Squared Error

Another popular loss function for regression models is the mean squared error (MSE), which is equal to $\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$. It is similar to the mean absolute error as it also measures the deviation of the predicted value from the ground truth value. However, the mean squared error squares this difference (always non-negative since squares of real numbers are always non-negative), which gives it slightly different properties.

One property is that the mean squared error favors a large number of small errors over a small number of large errors, which leads to models with fewer outliers or at least outliers that are less severe than models trained with a mean absolute error. This is because a large error would have a significantly larger impact on the error and, consequently, the gradient of the error when compared to a small error. Graphically,

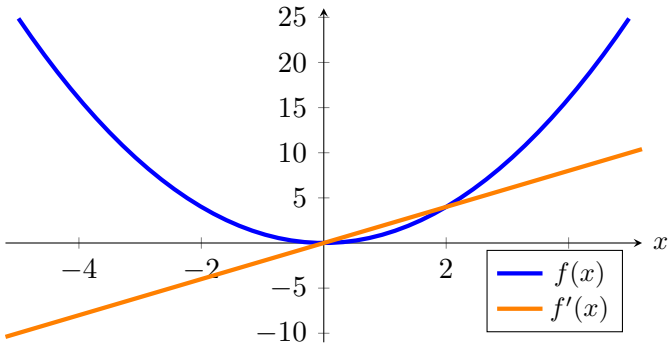

Figure 19.2: Mean square error loss function (blue) and gradient (orange)

Notice that larger errors would lead to a larger magnitude for the gradient and a larger loss. Hence, for example, two training examples that deviate from their ground truths by 1 unit would lead to a loss of 2, while a single training example that deviates from its ground truth by 2 units would lead to a loss of 4, hence having a larger impact.

Let's look at how to implement the mean squared loss in TensorFlow.

```
import tensorflow as tf
from tensorflow.keras.losses import MeanSquaredError

y_true = [1., 0.]
y_pred = [2., 3.]

mse_loss = MeanSquaredError()

print(mse_loss(y_true, y_pred).numpy())
```

Listing 19.2: Compute the mean squared error

This gives the output `5.0` as expected since $\frac{1}{2}[(2-1)^2 + (3-0)^2] = \frac{1}{2}(10) = 5$. Notice that the second example with a predicted value of 3 and actual value of 0 contributes 90% of the error under the mean squared error vs. 75% under the mean absolute error.

Sometimes, you may see people use root mean squared error (RMSE) as a metric. This will take the square root of MSE. From the perspective of a loss function, MSE and RMSE are equivalent.

Both MAE and MSE measure values in a continuous range. Hence they are for regression problems. For classification problems, you can use categorical cross-entropy.

19.4 Categorical Cross-Entropy

The previous two loss functions are for regression models, where the output could be any real number. However, for classification problems, there is a small, discrete set of numbers that the output could take. Furthermore, the number used to label-encode the classes is arbitrary and with no semantic meaning (e.g., using the labels 0 for cat, 1 for dog, and 2 for horse does not represent that a dog is half cat and half horse). Therefore, it should not have an impact on the performance of the model.

In a classification problem, the model's output is a vector of probability for each category. In Keras models, this vector is usually expected to be "logits," i.e., real numbers to be transformed to probability using the softmax function, or the output of a softmax activation function.

The cross-entropy between two probability distributions is a measure of the difference between the two probability distributions. Precisely, it is $-\sum_i P(X = x_i) \log Q(X = x_i)$ for probability P and Q. In machine learning, we usually have the probability P provided by the training data and Q predicted by the model, which P is 1 for the correct class and 0 for every other class. The predicted probability Q, however, is usually valued between 0 and 1. Hence when used for classification problems in machine learning, this formula can be simplified into:

$$\text{categorical cross-entropy} = -\log p_{gt}$$

where p_{gt} is the model-predicted probability of the ground truth class for that particular sample.

Cross-entropy metrics have a negative sign because $\log(x)$ tends to negative infinity as x tends to zero. We want a higher loss when the probability approaches 0 and a lower loss when the probability approaches 1. Graphically,

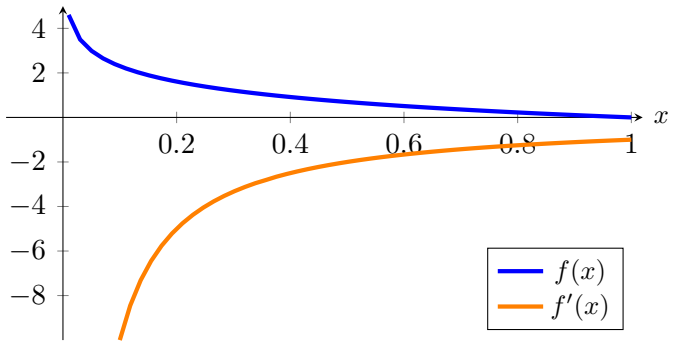

Figure 19.3: Categorical cross-entropy loss function (blue) and gradient (orange)

Notice that the loss is exactly 0 if the probability of the ground truth class is 1 as desired. Also, as the probability of the ground truth class tends to 0, the loss tends to positive infinity as well, hence substantially penalizing bad predictions. You might recognize this loss function for logistic regression, which is similar except the logistic regression loss is specific to the case of binary classes.

Looking at the gradient, you can see that the gradient is generally negative, which is also expected since, to decrease this loss, you would want the probability on the ground truth class to be as high as possible. Recall that gradient descent goes in the opposite direction of the gradient.

There are two different ways to implement categorical cross-entropy in TensorFlow. The first method takes in one-hot vectors as input,

```
import tensorflow as tf
from tensorflow.keras.losses import CategoricalCrossentropy

# using one-hot vector representation
y_true = [[0, 1, 0], [1, 0, 0]]
y_pred = [[0.15, 0.75, 0.1], [0.75, 0.15, 0.1]]

cross_entropy_loss = CategoricalCrossentropy()

print(cross_entropy_loss(y_true, y_pred).numpy())
```

Listing 19.3: Compute the categorical cross-entropy

This gives the output as `0.2876821` which is equal to $-\log(0.75)$ as expected. The other way of implementing the categorical cross-entropy loss in TensorFlow is using a label-encoded representation for the class, where the class is represented by a single non-negative integer indicating the ground truth class instead.

```
import tensorflow as tf
from tensorflow.keras.losses import SparseCategoricalCrossentropy

y_true = [1, 0]
y_pred = [[0.15, 0.75, 0.1], [0.75, 0.15, 0.1]]

cross_entropy_loss = SparseCategoricalCrossentropy()
```

```
print(cross_entropy_loss(y_true, y_pred).numpy())
```

Listing 19.4: Compute the sparse cross-entropy

This likewise gives the output `0.2876821`.

Now that you've explored loss functions for both regression and classification models, let's take a look at how you can use loss functions in your machine learning models.

19.5 Loss Functions in Practice

Let's explore how to use loss functions in practice. You'll explore this through a simple dense model on the MNIST digit classification dataset.

First, download the data from the Keras datasets module:

```
import tensorflow as tf

(trainX, trainY), (testX, testY) = tf.keras.datasets.mnist.load_data()
```

Listing 19.5: Getting the MNIST dataset

Then, build your model:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten

model = Sequential([
          Input(shape=(28,28,1,)),
          Flatten(),
          Dense(units=84, activation="relu"),
          Dense(units=10, activation="softmax"),
      ])

print (model.summary())
```

Listing 19.6: Building a model for MNIST dataset

And look at the model architecture outputted from the above code:

```
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 flatten_1 (Flatten)          (None, 784)               0

 dense_2 (Dense)              (None, 84)                65940

 dense_3 (Dense)              (None, 10)                850

=================================================================
Total params: 66,790
Trainable params: 66,790
```

```
Non-trainable params: 0
_________________________________________________________________
```

Output 19.1: The model architecture

You can then compile your model, which is also where you introduce the loss function. Since this is a classification problem, use the cross-entropy loss. In particular, since the MNIST dataset in Keras datasets is represented as a label instead of a one-hot vector, use the `SparseCategoricalCrossEntropy` loss.

```
model.compile(optimizer="adam",
              loss=tf.keras.losses.SparseCategoricalCrossentropy(), metrics="acc")
```

Listing 19.7: Setting loss function to a model

And finally, you train your model, with the complete code below:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten

(trainX, trainY), (testX, testY) = tf.keras.datasets.mnist.load_data()

model = Sequential([
          Input(shape=(28,28,1,)),
          Flatten(),
          Dense(units=84, activation="relu"),
          Dense(units=10, activation="softmax"),
      ])

model.summary()

model.compile(optimizer="adam",
              loss=tf.keras.losses.SparseCategoricalCrossentropy(), metrics="acc")

history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
                    validation_data=(testX, testY))
```

Listing 19.8: Build a train a model using sparse cross-entropy loss function

And your model successfully trains with the following output:

```
Epoch 1/10
235/235 [==============================] - 2s 6ms/step - loss: 7.8607 - acc: 0.8184 - val_loss: 1.74
Epoch 2/10
235/235 [==============================] - 1s 6ms/step - loss: 1.1011 - acc: 0.8854 - val_loss: 0.90
Epoch 3/10
235/235 [==============================] - 1s 6ms/step - loss: 0.5729 - acc: 0.8998 - val_loss: 0.66
Epoch 4/10
235/235 [==============================] - 1s 5ms/step - loss: 0.3911 - acc: 0.9203 - val_loss: 0.54
Epoch 5/10
235/235 [==============================] - 1s 6ms/step - loss: 0.3016 - acc: 0.9306 - val_loss: 0.50
Epoch 6/10
235/235 [==============================] - 1s 6ms/step - loss: 0.2443 - acc: 0.9405 - val_loss: 0.45
```

```
Epoch 7/10
235/235 [==============================] - 1s 5ms/step - loss: 0.2076 - acc: 0.9469 - val_loss: 0.41
Epoch 8/10
235/235 [==============================] - 1s 5ms/step - loss: 0.1852 - acc: 0.9514 - val_loss: 0.43
Epoch 9/10
235/235 [==============================] - 1s 6ms/step - loss: 0.1576 - acc: 0.9577 - val_loss: 0.42
Epoch 10/10
235/235 [==============================] - 1s 5ms/step - loss: 0.1455 - acc: 0.9597 - val_loss: 0.41
```

Output 19.2: Output of a model trained using the sparse cross-entropy loss function

And that's one example of how to use a loss function in a TensorFlow model.

19.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

APIs

Losses. Keras API Reference.
https://keras.io/api/losses/

tf.keras.losses.MeanAbsoluteError. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/MeanAbsoluteError

tf.keras.losses.MeanSquaredError. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/MeanSquaredError

tf.keras.losses.CategoricalCrossentropy. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/CategoricalCrossentropy

tf.keras.losses.SparseCategoricalCrossentropy. TensorFlow API.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/SparseCategoricalCrossentropy

19.7 Summary

In this chapter, you have seen loss functions and the role that they play in a neural network. You have also seen some popular loss functions used in regression and classification models, as well as how to use the cross-entropy loss function in a TensorFlow model.

Specifically, you learned:

- ▷ What are loss functions, and how they are different from metrics
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your TensorFlow model

In the next chapter, we will learn about regularization.

Reduce Overfitting with Dropout Regularization

20

Dropout is a simple and powerful regularization technique for neural networks and deep learning models. The goal is to prevent overfitting, which is when the network know too much about the training data that became ineffective to the validation data that it never saw. In this chapter, you will discover the dropout regularization technique and how to apply it to your models in Python with Keras. After reading this chapter, you will know:

- ▷ How the dropout regularization technique works
- ▷ How to use dropout on your input layers
- ▷ How to use dropout on your hidden layers
- ▷ How to tune the dropout level on your problem

Let's get started.

Overview

This chapter is divided into six parts; they are:

- ▷ Dropout Regularization For Neural Networks
- ▷ Dropout Regularization in Keras
- ▷ Using Dropout on the Visible Layer
- ▷ Using Dropout on Hidden Layers
- ▷ Dropout in Evaluation Mode
- ▷ Tips For Using Dropout

20.1 Dropout Regularization For Neural Networks

Dropout is a regularization technique for neural network models proposed by Srivastava et al. in their 2014 paper "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". Dropout is a technique where randomly selected neurons are ignored during training. They are *dropped out* randomly. This means that their contribution to the activation of downstream

neurons is temporally removed on the forward pass, and any weight updates are not applied to the neuron on the backward pass.

As a neural network learns, neuron weights settle into their context within the network. Weights of neurons are tuned for specific features, providing some specialization. Neighboring neurons come to rely on this specialization, which, if taken too far, can result in a fragile model too specialized for the training data, i.e., overfitting. This reliance context for a neuron during training is referred to as *complex co-adaptations*. You can imagine that if neurons are randomly dropped out of the network during training, other neurons will have to step in and handle the representation required to make predictions for the missing neurons. This is believed to result in multiple independent internal representations being learned by the network.

The effect is that the network becomes less sensitive to the specific weights of neurons. This, in turn, results in a network capable of better generalization and less likely to overfit the training data.

20.2 Dropout Regularization in Keras

Dropout is easily implemented by randomly selecting nodes to be dropped out with a given probability (e.g., 20%) in each weight update cycle. This is how Dropout is implemented in Keras. Dropout is only used during the training of a model and is not used when evaluating the skill of the model. Next, let's explore a few different ways of using Dropout in Keras.

The examples will use the Sonar binary classification dataset (see Section 10.1). You will evaluate the developed models using scikit-learn with 10-fold cross-validation in order to tease out differences in the results better. There are 60 input values and a single output value. The input values are standardized before being used in the network. The baseline neural network model has two hidden layers, the first with 60 units and the second with 30. Stochastic gradient descent is used to train the model with a relatively low learning rate and momentum.

The full baseline model is listed below:

```
# Baseline Model on the Sonar Dataset
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
```

```
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)

# baseline
def create_baseline():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(30,  activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(learning_rate=0.01, momentum=0.8)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model

estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_baseline,
                                          epochs=300, batch_size=16, verbose=0)))
pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Baseline: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 20.1: Baseline neural network for the sonar dataset

Running the example for the baseline model generates an estimated classification accuracy of 86%.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Baseline: 86.04% (4.58%)
```

Output 20.1: Sample output from baseline neural network for the sonar dataset

20.3 Using Dropout on the Visible Layer

Dropout can be applied to input neurons called the visible layer. In the example below, a new `Dropout` layer between the input (or visible layer) and the first hidden layer was added. The dropout rate is set to 20%, meaning one in five inputs will be randomly excluded from each update cycle. In the example of the sonar dataset, using dropout means the prediction is based on only 80% of the input features (randomly chosen).

Additionally, as recommended in the original paper on dropout, a constraint is imposed on the weights for each hidden layer, ensuring that the maximum norm of the weights does not

exceed a value of 3. This is done by setting the `kernel_constraint` argument on the `Dense` class when constructing the layers. The learning rate was lifted by one order of magnitude, and the momentum was increased to 0.9. These increases in the learning rate were also recommended in the original dropout paper. Continuing from the baseline example above, the code below exercises the same network with input dropout:

```
# Example of Dropout on the Sonar Dataset: Visible Layer
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.optimizers import SGD
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)

# dropout in the input layer with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dropout(0.2, input_shape=(60,)))
    model.add(Dense(60, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dense(30, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(learning_rate=0.1, momentum=0.9)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model

estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_model,
                                          epochs=300, batch_size=16, verbose=0)))
pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Visible: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 20.2: Example of using dropout on the visible layer

Running the example provides a slight drop in classification accuracy, at least on a single test run.

```
Visible: 83.52% (7.68%)
```

Output 20.2: Sample output from example of using dropout on the visible layer

20.4 Using Dropout on Hidden Layers

Dropout can be applied to hidden neurons in the body of your network model. In the example below, dropout is applied between the two hidden layers and between the last hidden layer and the output layer. Again a dropout rate of 20% is used as is a weight constraint on those layers. In the example of the sonar dataset, this means the prediction is based on only 80% of the features learned by the hidden layer (randomly chosen).

```
# Example of Dropout on the Sonar Dataset: Hidden Layer
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.optimizers import SGD
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)

# dropout in hidden layers with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,),
                    activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(30, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(learning_rate=0.1, momentum=0.9)
```

```
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model

estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_model,
                                          epochs=300, batch_size=16, verbose=0)))
pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Hidden: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 20.3: Example of using dropout on the hidden layer

You can see that for this problem and the chosen network configuration, using dropout in the hidden layers did not lift performance. In fact, performance was worse than the baseline. It is possible that additional training epochs are required or that further tuning is required to the learning rate.

```
Hidden: 83.59% (7.31%)
```

Output 20.3: Sample output from example of using dropout on the hidden layer

20.5 Dropout in Evaluation Mode

Dropout will randomly reset some of the input to zero. If you wonder what happens after you have finished training, the answer is nothing! In Keras, a layer can tell if the model is running in training mode or not. The `Dropout` layer will randomly reset some input only when the model runs for training. Otherwise, the `Dropout` layer works as a scaler to multiply all input by a factor such that the next layer will see input similar in scale. Precisely, if the dropout rate is r, the input will be scaled by a factor of $1 - r$.

20.6 Tips For Using Dropout

The original paper on Dropout provides experimental results on a suite of standard machine learning problems. As a result, they provide a number of useful heuristics to consider when using dropout in practice:

- ▷ Generally, use a small dropout value of 20%–50% of neurons, with 20% providing a good starting point. A probability too low has minimal effect, and a value too high results in under-learning by the network.
- ▷ Use a larger network. You are likely to get better performance when dropout is used on a larger network, giving the model more of an opportunity to learn independent representations.
- ▷ Use dropout on input (visible) as well as hidden layers. Application of dropout at each layer of the network has shown good results.

- ▷ Use a large learning rate with decay and a large momentum. Increase your learning rate by a factor of 10 to 100 and use a high momentum value of 0.9 or 0.99.
- ▷ Constrain the size of network weights. A large learning rate can result in very large network weights. Imposing a constraint on the size of network weights, such as max-norm regularization, with a size of 4 or 5 has been shown to improve results.

20.7 More Resources on Dropout

This section provides more resources on the topic if you are looking to go deeper.

Articles

Nitish Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". *Journal of Machine Learning Research*, 15(56), 2014, pp. 1929–1958.
https://jmlr.org/papers/v15/srivastava14a.html

Geoffrey E. Hinton et al. "Improving neural networks by preventing co-adaptation of feature detectors", 2012.
https://arxiv.org/abs/1207.0580

How does the dropout method work in deep learning? Quora.
https://www.quora.com/How-does-the-dropout-method-work-in-deep-learning

Keras Training and Evaluation with Built-in Methods. TensorFlow documentation.
https://www.tensorflow.org/guide/keras/train_and_evaluate

20.8 Summary

In this chapter, you discovered the dropout regularization technique for deep learning models. You learned:

- ▷ What dropout is and how it works
- ▷ How you can use dropout on your own deep learning models
- ▷ Tips for getting the best results from dropout on your own models

In the next chapter, you will learn how to change the learning rate during training.

Lift Performance with Learning Rate Schedules

21

Training a neural network or large deep learning model is a difficult optimization task. The classical algorithm to train neural networks is called stochastic gradient descent. It has been well established that you can achieve increased performance and faster training on some problems by using a learning rate that changes during training. In this chapter, you will discover how you can use different learning rate schedules for your neural network models in Python using the Keras deep learning library. After completing this chapter, you will know:

- ▷ The benefit of learning rate schedules on lifting model performance during training
- ▷ How to configure and evaluate a time-based learning rate schedule
- ▷ How to configure and evaluate a drop-based learning rate schedule

Let's get started.

Overview

This chapter is divided into four parts; they are:

- ▷ Learning Rate Schedule For Training Models
- ▷ Time-Based Learning Rate Schedule
- ▷ Drop-Based Learning Rate Schedule
- ▷ Tips for Using Learning Rate Schedules

21.1 Learning Rate Schedule for Training Models

Adapting the learning rate for your stochastic gradient descent optimization procedure can increase performance and reduce training time. Sometimes, this is called learning rate annealing or adaptive learning rates. Here, this approach is called a learning rate schedule, where the default schedule usesr a constant learning rate to update network weights for each training epoch.

The simplest and perhaps most used adaptation of the learning rates during training are techniques that reduce the learning rate over time. These have the benefit of making large

changes at the beginning of the training procedure when larger learning rate values are used and decreasing the learning rate so that a smaller rate and, therefore, smaller training updates are made to weights later in the training procedure. This has the effect of quickly learning good weights early and fine-tuning them later. Two popular and easy-to-use learning rate schedules are as follows:

- ▷ Decrease the learning rate gradually based on the epoch
- ▷ Decrease the learning rate using punctuated large drops at specific epochs

Next, let's look at how you can use each of these learning rate schedules in turn with Keras.

21.2 Time-Based Learning Rate Schedule

Keras has a built-in time-based learning rate schedule. The stochastic gradient descent optimization algorithm implementation in the `SGD` class has an argument called decay. This argument is used in the time-based learning rate decay schedule equation as follows:

$$\text{LearningRate} := \text{LearningRate} \times \frac{1}{1 + \text{decay} \times \text{epoch}}$$

When the decay argument is zero (the default), this does not affect the learning rate. For example, when the learning rate was 0.1,

$$\text{LearningRate} := 0.1 \times \frac{1}{1 + 0 \times \text{epoch}} = 0.1$$

When the decay argument is specified, it will decrease the learning rate from the previous epoch by the given fixed amount. For example, if you use the initial learning rate value of 0.1 and the decay of 0.001, the first five epochs will adapt the learning rate as follows:

```
Epoch    Learning Rate
1        0.1
2        0.0999000999
3        0.0997006985
4        0.09940249103
5        0.09900646517
```

Output 21.1: Learning rate with decay as calculated

Extending this out to 100 epochs will produce the following graph of learning rate (y-axis) versus epoch (x-axis):

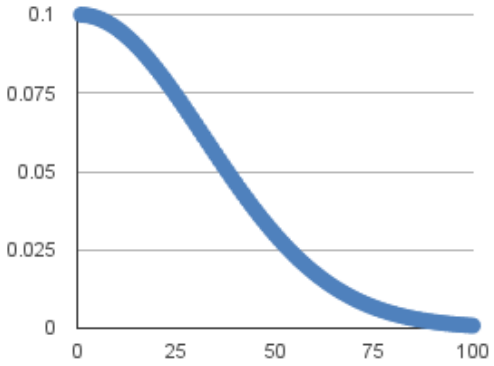

Figure 21.1: Time-based learning rate schedule

You can create a nice default schedule by setting the decay value as follows:

$$\text{Decay} = \frac{\text{LearningRate}}{\text{Epochs}} = \frac{0.1}{100} = 0.001$$

The example below demonstrates using the time-based learning rate adaptation schedule in Keras.

It is demonstrated in the Ionosphere binary classification problem. The ionosphere dataset is good for practicing with neural networks because all the input values are small numerical values of the same scale. The dataset describes radar returns where the target was free electrons in the ionosphere. It is a binary classification problem where positive cases (`g` for good) show evidence of some type of structure in the ionosphere and negative cases (`b` for bad) do not. It is a good dataset for practicing with neural networks because all of the inputs are small numerical values of the same scale. There are 34 attributes and 351 observations.

State-of-the-art results on this dataset achieve an accuracy of approximately 94% to 98% accuracy using 10-fold cross-validation. The dataset is available within the code bundle provided with this book. Alternatively, you can download it directly from the UCI Machine Learning repository[1]. Place the data file in your working directory with the filename `ionosphere.csv`. You can learn more about the ionosphere dataset on the UCI Machine Learning Repository website.

A small neural network model is constructed with a single hidden layer with 34 neurons, using the rectifier activation function. The output layer has a single neuron and uses the sigmoid activation function in order to output probability-like values. The learning rate for stochastic gradient descent has been set to a higher value of 0.1. The model is trained for 50 epochs, and the decay argument has been set to 0.002, calculated as $\frac{0.1}{50}$. Additionally, it can be a good idea to use momentum when using an adaptive learning rate. In this case, we use a momentum value of 0.8. The complete example is listed below.

[1] https://raw.githubusercontent.com/jbrownlee/Datasets/master/ionosphere.csv

```
# Time Based Learning Rate Decay
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from sklearn.preprocessing import LabelEncoder
# load dataset
dataframe = read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:34].astype(float)
Y = dataset[:,34]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
Y = encoder.transform(Y)
# create model
model = Sequential()
model.add(Dense(34, input_shape=(34,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
epochs = 50
learning_rate = 0.1
decay_rate = learning_rate / epochs
momentum = 0.8
sgd = SGD(learning_rate=learning_rate, momentum=momentum, decay=decay_rate,
          nesterov=False)
model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=epochs, batch_size=28, verbose=2)
```

Listing 21.1: Example of time-based learning rate decay

The model is trained on 67% of the dataset and evaluated using a 33% validation dataset. Running the example shows a classification accuracy of 99.14%. This is higher than the baseline of 95.69% without the learning rate decay or momentum.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
...
Epoch 45/50
0s - loss: 0.0622 - acc: 0.9830 - val_loss: 0.0929 - val_acc: 0.9914
Epoch 46/50
0s - loss: 0.0695 - acc: 0.9830 - val_loss: 0.0693 - val_acc: 0.9828
Epoch 47/50
0s - loss: 0.0669 - acc: 0.9872 - val_loss: 0.0616 - val_acc: 0.9828
Epoch 48/50
0s - loss: 0.0632 - acc: 0.9830 - val_loss: 0.0824 - val_acc: 0.9914
Epoch 49/50
0s - loss: 0.0590 - acc: 0.9830 - val_loss: 0.0772 - val_acc: 0.9828
```

```
Epoch 50/50
0s - loss: 0.0592 - acc: 0.9872 - val_loss: 0.0639 - val_acc: 0.9828
```

Output 21.2: Sample output of time-based learning rate decay

21.3 Drop-Based Learning Rate Schedule

Another popular learning rate schedule used with deep learning models is systematically dropping the learning rate at specific times during training. Often this method is implemented by dropping the learning rate by half every fixed number of epochs. For example, we may have an initial learning rate of 0.1 and drop it by a factor of 0.5 every ten epochs. The first ten epochs of training would use a value of 0.1, and in the next ten epochs, a learning rate of 0.05 would be used, and so on. If we plot out the learning rates for this example out to 100 epochs, you get the graph below showing the learning rate (y-axis) versus epoch (x-axis).

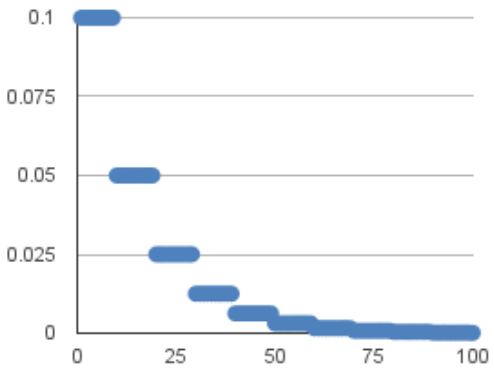

Figure 21.2: Drop-based learning rate schedule

You can implement this in Keras using the `LearningRateScheduler` callback when fitting the model. The `LearningRateScheduler` callback allows you to define a function to call that takes the epoch number as an argument and returns the learning rate to use in stochastic gradient descent. When used, the learning rate specified by stochastic gradient descent is ignored. In the code below, we use the same example as before of a single hidden layer network on the Ionosphere dataset. A new `step_decay()` function is defined that implements the equation:

$$\text{LearningRate} = \text{InitialLearningRate} \times \text{DropRate}^{\text{floor}(\frac{1+\text{Epoch}}{\text{EpochDrop}})}$$

Here, the InitialLearningRate is the initial learning rate (such as 0.1), the DropRate is the amount that the learning rate is modified each time it is changed (such as 0.5), Epoch is the current epoch number, and EpochDrop is how often to change the learning rate (such as 10). Notice that the learning rate in the SGD class is set to 0 to clearly indicate that it is not used. Nevertheless, you can set a momentum term in SGD if you want to use momentum with this learning rate schedule.

```
# Drop-Based Learning Rate Decay
from pandas import read_csv
import math
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.callbacks import LearningRateScheduler

# learning rate schedule
def step_decay(epoch):
    initial_lrate = 0.1
    drop = 0.5
    epochs_drop = 10.0
    lrate = initial_lrate * math.pow(drop, math.floor((1+epoch)/epochs_drop))
    return lrate

# load dataset
dataframe = read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:34].astype(float)
Y = dataset[:,34]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
Y = encoder.transform(Y)
# create model
model = Sequential()
model.add(Dense(34, input_shape=(34,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
sgd = SGD(learning_rate=0.0, momentum=0.9)
model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
# learning schedule callback
lrate = LearningRateScheduler(step_decay)
callbacks_list = [lrate]
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=50, batch_size=28,
          callbacks=callbacks_list, verbose=2)
```

Listing 21.2: Example of drop-based learning rate decay

Running the example results in a classification accuracy of 99.14% on the validation dataset, again an improvement over the baseline for the model of this dataset.

```
...
Epoch 45/50
0s - loss: 0.0546 - acc: 0.9830 - val_loss: 0.0634 - val_acc: 0.9914
Epoch 46/50
0s - loss: 0.0544 - acc: 0.9872 - val_loss: 0.0638 - val_acc: 0.9914
Epoch 47/50
0s - loss: 0.0553 - acc: 0.9872 - val_loss: 0.0696 - val_acc: 0.9914
Epoch 48/50
```

```
0s - loss: 0.0537 - acc: 0.9872 - val_loss: 0.0675 - val_acc: 0.9914
Epoch 49/50
0s - loss: 0.0537 - acc: 0.9872 - val_loss: 0.0636 - val_acc: 0.9914
Epoch 50/50
0s - loss: 0.0534 - acc: 0.9872 - val_loss: 0.0679 - val_acc: 0.9914
```

Output 21.3: Sample output of drop-based learning rate decay

21.4 Tips for Using Learning Rate Schedules

This section lists some tips and tricks to consider when using learning rate schedules with neural networks.

- ▷ **Increase the initial learning rate**. Because the learning rate will decrease, start with a larger value to decrease from. A larger learning rate will result in much larger changes to the weights, at least in the beginning, allowing you to benefit from fine-tuning later.
- ▷ **Use a larger momentum**. Larger momentum value will make the update carry on to later steps. It helps the optimization algorithm continue to make updates in the right direction when your learning rate shrinks to small values.
- ▷ **Experiment with different schedules**. It will not be clear which learning rate schedule to use, so try a few with different configuration options and see what works best on your problem. Also, try schedules that change exponentially and even schedules that respond to the accuracy of your model on the training or test datasets.

21.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Ionosphere Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Ionosphere

21.6 Summary

In this chapter, you discovered learning rate schedules for training neural network models. You learned:

- ▷ The benefits of using learning rate schedules during training to lift model performance
- ▷ How to configure and use a time-based learning rate schedule in Keras
- ▷ How to develop your own drop-based learning rate schedule in Keras

In the next chapter, you will learn about different ways to feed training data to a Keras model.

Introduction to tf.data API

22

When you build and train a Keras deep learning model, you can provide the training data in several different ways. Presenting the data as a NumPy array or a TensorFlow tensor is common. Another way is to make a Python generator function and let the training loop read data from it. Yet another way of providing data is to use `tf.data` dataset.

In this chapter, you will see how you can use the `tf.data` dataset for a Keras model. After finishing this chapter, you will learn:

- ▷ How to create and use the `tf.data` dataset
- ▷ The benefit of doing so compared to a generator function

Let's get started.

Overview

This chapter is divided into four sections; they are:

- ▷ Training a Keras Model with NumPy Array and Generator Function
- ▷ Creating a Dataset using `tf.data`
- ▷ Creating a Dataest from Generator Function
- ▷ Data with Prefetch

22.1 Training a Keras Model with NumPy Array and Generator Function

Before you see how the `tf.data` API works, let's review how you usually train a Keras model.

First, you need a dataset. An example is the Fashion-MNIST dataset that comes with the Keras API. This dataset has 60,000 training samples and 10,000 test samples of 28×28 pixels in grayscale, and the corresponding classification label is encoded with integers 0 to 9. The dataset is a NumPy array. Then you can build a Keras model for classification, and with the model's `fit()` function, you provide the NumPy array as data.

The complete code is as follows:

```
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets.fashion_mnist import load_data
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential

(train_image, train_label), (test_image, test_label) = load_data()
print(train_image.shape)
print(train_label.shape)
print(test_image.shape)
print(test_label.shape)

model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(100, activation="relu"),
    Dense(100, activation="relu"),
    Dense(10, activation="sigmoid")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")
history = model.fit(train_image, train_label,
                    batch_size=32, epochs=50,
                    validation_data=(test_image, test_label), verbose=0)
print(model.evaluate(test_image, test_label))

plt.plot(history.history['val_sparse_categorical_accuracy'])
plt.show()
```

Listing 22.1: Training a neural network with dataset in NumPy arrays

Running this code will plot the validation accuracy over the 50 epochs you trained your model:

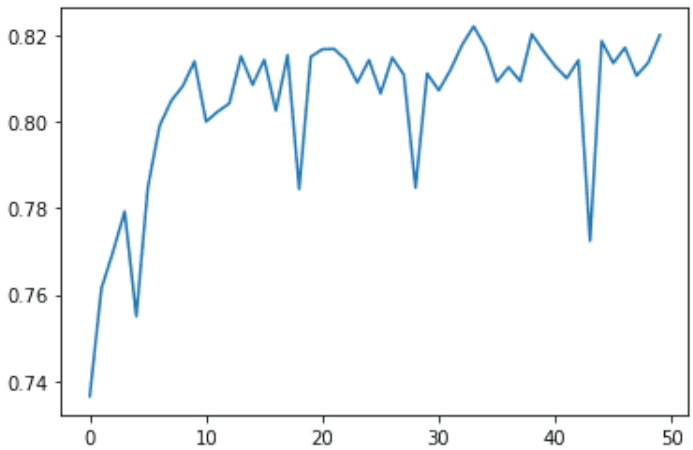

Figure 22.1: Validation accuracy achieved during 50 epochs of training

And also, print out the matrices' shape and the evaluation score:

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
313/313 [==============================] - 0s 392us/step - loss: 0.5114 - sparse_categoric
[0.5113903284072876, 0.8446000218391418]
```

Output 22.1: Screen output of training with dataset in NumPy array

The other way of training the same network is to provide the data from a Python generator function instead of a NumPy array. A generator function is the one with a `yield` statement to emit data while the function runs in parallel to the data consumer. A generator of the Fashion-MNIST dataset can be created as follows:

```
def batch_generator(image, label, batchsize):
    N = len(image)
    i = 0
    while True:
        yield image[i:i+batchsize], label[i:i+batchsize]
        i = i + batchsize
        if i + batchsize > N:
            i = 0
```

Listing 22.2: A batch generator for training a Keras model

This function is supposed to be called with the syntax

```
batch_generator(train_image, train_label, 32)
```

It will scan the input arrays in batches indefinitely. Once it reaches the end of the array, it will restart from the beginning.

Training a Keras model with a generator is similar to using the `fit()` function:

```
history = model.fit(batch_generator(train_image, train_label, 32),
                    steps_per_epoch=len(train_image)//32,
                    epochs=50, validation_data=(test_image, test_label), verbose=0)
```

Listing 22.3: Training a Keras model with a generator

Instead of providing the data and label, you just need to provide the generator as it will give out both. When data are presented as a NumPy array, you can tell how many samples there are by looking at the length of the array. Keras can complete one epoch when the entire dataset is used once. However, your generator function will emit batches indefinitely, so you need to tell it when an epoch is ended, using the `steps_per_epoch` argument to the `fit()` function.

In the above code, the validation data was provided as a NumPy array, but you can also use a generator instead and specify the `validation_steps` argument.

The following is the complete code using a generator function, in which the output is the same as the previous example:

```
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets.fashion_mnist import load_data
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential

model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(100, activation="relu"),
    Dense(100, activation="relu"),
    Dense(10, activation="sigmoid")
])

def batch_generator(image, label, batchsize):
    N = len(image)
    i = 0
    while True:
        yield image[i:i+batchsize], label[i:i+batchsize]
        i = i + batchsize
        if i + batchsize > N:
            i = 0

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")
history = model.fit(batch_generator(train_image, train_label, 32),
                    steps_per_epoch=len(train_image)//32,
                    epochs=50, validation_data=(test_image, test_label), verbose=2)
print(model.evaluate(test_image, test_label))

plt.plot(history.history['val_sparse_categorical_accuracy'])
plt.show()
```

Listing 22.4: Complete example of using a generator function to train a Keras model

22.2 Creating a Dataset Using tf.data

Given that you have the Fashion-MNIST data loaded, you can convert it into a `tf.data` dataset, like the following:

```
...
dataset = tf.data.Dataset.from_tensor_slices((train_image, train_label))
print(dataset.element_spec)
```

Listing 22.5: Creating a `tf.data` dataset

This prints the dataset's spec as follows:

```
(TensorSpec(shape=(28, 28), dtype=tf.uint8, name=None),
 TensorSpec(shape=(), dtype=tf.uint8, name=None))
```

Output 22.2: Data spec of a `tf.data` dataset

You can see the data is a tuple (as a tuple was passed as an argument to the `from_tensor_slices()` function), whereas the first element is in the shape `(28,28)` while the second element is a scalar. Both elements are stored as 8-bit unsigned integers.

If you do not present the data as a tuple of two NumPy arrays when you create the dataset, you can also do it later. The following creates the same dataset but first creates the dataset for the image data and the label separately before combining them:

```
...
train_image_data = tf.data.Dataset.from_tensor_slices(train_image)
train_label_data = tf.data.Dataset.from_tensor_slices(train_label)
dataset = tf.data.Dataset.zip((train_image_data, train_label_data))
print(dataset.element_spec)
```

Listing 22.6: Creating a dataset by combining two other datasets

This will print the same spec:

```
(TensorSpec(shape=(28, 28), dtype=tf.uint8, name=None),
 TensorSpec(shape=(), dtype=tf.uint8, name=None))
```

Output 22.3: Data spec of a `tf.data` dataset

The `zip()` function in the dataset is like the `zip()` function in Python because it matches data one by one from multiple datasets into a tuple.

One benefit of using the `tf.data` dataset is the flexibility in handling the data. Below is the complete code on how you can train a Keras model using a dataset in which the batch size is set to the dataset:

```
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets.fashion_mnist import load_data
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential

(train_image, train_label), (test_image, test_label) = load_data()
dataset = tf.data.Dataset.from_tensor_slices((train_image, train_label))

model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(100, activation="relu"),
    Dense(100, activation="relu"),
    Dense(10, activation="sigmoid")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")
history = model.fit(dataset.batch(32),
                    epochs=50, validation_data=(test_image, test_label), verbose=2)
print(model.evaluate(test_image, test_label))

plt.plot(history.history['val_sparse_categorical_accuracy'])
plt.show()
```

Listing 22.7: Training a neural network with `tf.data` dataset

This is the simplest use case of using a dataset. If you dive deeper, you can see that a dataset is just an iterator. Therefore, you can print out each sample in a dataset using the following:

```
for image, label in dataset:
    print(image)  # array of shape (28,28) in tf.Tensor
    print(label)  # integer label in tf.Tensor
```

Listing 22.8: Iteratively extracting samples from a dataset

The dataset has many functions built in. The `batch()` used before is one of them. If you create batches from dataset and print them, you have the following:

```
for image, label in dataset.batch(32):
    print(image)  # array of shape (32,28,28) in tf.Tensor
    print(label)  # array of shape (32,) in tf.Tensor
```

Listing 22.9: Iteratively extracting batchs from a dataset

Here, each item from a batch is not a sample but a batch of samples. You also have functions such as `map()`, `filter()`, and `reduce()` for sequence transformation, or `concatendate()` and `interleave()` for combining with another dataset. There are also `repeat()`, `take()`, `take_while()`, and `skip()` like our familiar counterpart from Python's `itertools` module. A full list of the functions can be found in the API documentation.

22.3 Creating a Dataset from Generator Function

So far, you saw how a dataset could be used in place of a NumPy array in training a Keras model. Indeed, a dataset can also be created out of a generator function. But instead of a generator function that generates a *batch*, as you saw in one of the examples above, you now make a generator function that generates one sample at a time. The following is the function:

```
import numpy as np
import tensorflow as tf

def shuffle_generator(image, label, seed):
    idx = np.arange(len(image))
    np.random.default_rng(seed).shuffle(idx)
    for i in idx:
        yield image[i], label[i]

dataset = tf.data.Dataset.from_generator(
    shuffle_generator,
    args=[train_image, train_label, 42],
    output_signature=(
        tf.TensorSpec(shape=(28,28), dtype=tf.uint8),
        tf.TensorSpec(shape=(), dtype=tf.uint8)))
print(dataset.element_spec)
```

Listing 22.10: Creating a dataset from a generator function

This function randomizes the input array by shuffling the index vector. Then it generates one sample at a time. Unlike the previous example, this generator will end when the samples from the array are exhausted.

You can create a dataset from the function using `from_generator()`. You need to provide the name of the generator function (instead of an instantiated generator) and also the output signature of the dataset. This is required because the `tf.data.Dataset` API cannot infer the dataset spec before the generator is consumed.

Running the above code will print the same spec as before:

```
(TensorSpec(shape=(28, 28), dtype=tf.uint8, name=None),
 TensorSpec(shape=(), dtype=tf.uint8, name=None))
```

Output 22.4: Data spec of a dataset created with the generator function

Such a dataset is functionally equivalent to the dataset that you created previously. Hence you can use it for training as before. The following is the complete code:

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets.fashion_mnist import load_data
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Sequential

(train_image, train_label), (test_image, test_label) = load_data()

def shuffle_generator(image, label, seed):
    idx = np.arange(len(image))
    np.random.default_rng(seed).shuffle(idx)
    for i in idx:
        yield image[i], label[i]

dataset = tf.data.Dataset.from_generator(
    shuffle_generator,
    args=[train_image, train_label, 42],
    output_signature=(
        tf.TensorSpec(shape=(28,28), dtype=tf.uint8),
        tf.TensorSpec(shape=(), dtype=tf.uint8)))

model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(100, activation="relu"),
    Dense(100, activation="relu"),
    Dense(10, activation="sigmoid")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics="sparse_categorical_accuracy")
history = model.fit(dataset.batch(32),
                    epochs=50, validation_data=(test_image, test_label), verbose=2)
print(model.evaluate(test_image, test_label))
```

```
plt.plot(history.history['val_sparse_categorical_accuracy'])
plt.show()
```

Listing 22.11: Training a neural network with `tf.data` dataset created using a generator

22.4 Dataset with Prefetch

The real benefit of using a dataset is to use `prefetch()`.

Using a NumPy array for training is probably the best in performance. However, this means you need to load all data into memory. Using a generator function for training allows you to prepare one batch at a time, in which the data can be loaded from disk on demand, for example. Nonetheless, using a generator function to train a Keras model means either the training loop or the generator function is running at any time. If you work on a computer vision problem which the data are images on disk, and you need to do some image preprocessing before applying to your model, you want to run gradient descent on one batch of image while preparing the next batch. It is not easy to make the generator function and Keras' training loop run in parallel.

Dataset is the API that allows the generator and the training loop to run in parallel. If you have a generator that is computationally expensive (e.g., doing image augmentation in realtime), you can create a dataset from such a generator function and then use it with `prefetch()`, as follows:

```
...
history = model.fit(dataset.batch(32).prefetch(3),
                    epochs=50,
                    validation_data=(test_image, test_label),
                    verbose=0)
```

Listing 22.12: Using `prefetch()` with a dataset

The number argument to `prefetch()` is the size of the buffer. Here, the dataset is asked to keep three batches in memory ready for the training loop to consume. Whenever a batch is consumed, the dataset API will resume the generator function to refill the buffer asynchronously in the background. Therefore, you can allow the training loop and the data preparation algorithm inside the generator function to run in parallel.

It's worth mentioning that, in the previous section, you created a shuffling generator for the dataset API. Indeed the dataset API also has a `shuffle()` function to do the same, but you may not want to use it unless the dataset is small enough to fit in memory.

The `shuffle()` function, same as `prefetch()`, takes a buffer size argument. The shuffle algorithm will fill the buffer with the dataset and draw one element randomly from it. The consumed element will be replaced with the next element from the dataset. Hence you need the buffer as large as the dataset itself to make a truly random shuffle. This limitation is demonstrated with the following snippet:

```
import tensorflow as tf
import numpy as np

n_dataset = tf.data.Dataset.from_tensor_slices(np.arange(10000))
shuffled = []
for n in n_dataset.shuffle(10).take(20):
    shuffled.append(n.numpy())
print(shuffled)
```

Listing 22.13: Using `shuffle()` with a small buffer

The output from the above looks like the following:

```
[7, 4, 1, 10, 13, 2, 6, 0, 3, 18, 8, 17, 15, 20, 22, 9, 12, 25, 21, 5]
```

Output 22.5: Data from a dataset shuffled with small buffer

Here you can see the numbers are shuffled around its neighborhood, and you never see large numbers from its output.

22.5 Further Reading

More about the `tf.data` dataset can be found from its API documentation:

`tf.data.Datasets`. TensorFlow API.
https://www.tensorflow.org/datasets

22.6 Summary

In this chapter, you have seen how you can use the `tf.data` dataset and how it can be used in training a Keras model. Specifically, you learned:

- How to train a model using data from a NumPy array, a generator, and a dataset
- How to create a dataset using a NumPy array or a generator function
- How to use prefetch with a dataset to make the generator and training loop run in parallel

Start from the next chapter, you will learn about a different kind of neural networks that applies to images.